

二、选题依据

批注 [r1]: 此报告基本达到概要设计要求。存在的主要问题是国内外概况和发展趋势中对技术的分析不够。

1. 阐述该选题的研究意义，或工程设计的价值和意义，国内外概况和发展趋势，选题的先进性和实用性，技术难度及工作量。

1.1 选题的研究意义

科技的进步使我们步入了信息时代,这个时代拥有 4G 网络、WLAN、蓝牙通信等无线通信技术以及可以媲美 PC 功能的各种智能移动终端。随着这些技术的飞速发展,我们的生活越来越便捷,信息数据的获取也几乎可以做到信手拈来;特别是随着无线互联技术的普及,我们生活中越来越离不开这些能给我们带来便捷的科技成果,它们甚至从金融、教育、物流、安防等特殊行业扩展到了一些传统行业如餐饮业中。2013 年中国的智能手机用户已达到 3.54 亿。随着智能手机的普及,越来越多的人通过手机完成其对餐饮、出行等各方面的需求。有道提供的研究数据显示,2012 年通过 App 查询信息的用户超过智能手机用户的 61%,其中占比最重的为餐饮。可以预见的是,移动订餐系统将会成为餐饮行业追随科技发展步伐中一个重要的改革点。

“民以食为天”,这句俗语道出了食物在我们日常生活中不可撼动的地位。而人们生活水平的提高,也带动了餐饮行业的发展,进而使我们国家的饮食文化发扬光大。在 2013 年,中国餐饮业实现收入 25392 亿 同比增长 9%,但大部分餐饮企业还是以传统的方式提供预约以及点餐服务。在客流量大的时候,往往服务员会无暇顾及每一个顾客,导致顾客满意度下降。因此有必要开发一套订餐系统,在就餐高峰期让用户可以提前完成点餐,省去了到餐厅后等待服务员接待以及点餐的时间,同时为用户和商家节约了成本。

本系统的核心实现是基于移动互联网的移动应用系统,随着 4G 网络逐步对中国所有县级城市实现覆盖、wifi 无线网络技术的成熟、数据套餐资费持续下降以及第四代移动通信的兴起,直接导致国内移动互联网用户数量的急剧增长,利用手机完成购物、购买电影票、下载优惠券、交友等已经普及,所以该系统也必将快速普及,能够为用户提供更好的移动订餐体验。

本移动互联网应用系统,其本身就具有随时随地性,用户可以随时随地的了解到周边餐厅的详情,方便用户选择合适的地点进行用餐,能够最大限度的促使停车设施利用的均衡化,另外,该系统配有的百度导航系统,可以最大限度的降低用户为寻找餐厅路线而附加产生的交通量。

1.2 国内外概况和发展趋势

◎ 订餐系统国内外发展现状

当前在国内所采用的订餐方式还是以电话等通讯作为主要形式，不可否认，电话这种订餐方式非常便捷，但是这种方式存在一些弊端和不足，比如：电话订餐的撤单率高，效率低下，所有相关订餐信息的存储及变更都需要人工干预，极易出错。因此目前基于互联网的网络订餐服务在逐渐兴起^[1]。根据目前各城市的现状来看，在天津、北京、济南等代表城市，消费者对订餐服务的需求量还是非常大的，比如天津，由于这里有全国最大的专业批发市场，大胡同商城等，每天的往来商客非常集中，由于行程密集而无法便捷的就餐，因而App订餐受到这部分人群的青睐，同时订餐平台还提供了许多免费附加增值服务，吸引力很大，总体来看，目前国内的App订餐业务还处于增长期^[2]。其中包括线上订购、在线支付等等。其实，想法都很不错，但想实现App订购，这些订餐的形式应该很难，原因主要就是餐饮行业普遍存在的信息化意识薄弱，这让App订餐成为实在的瓶颈，短时期内这是很难突破的，这也是为什么现在真正能够实现App订餐、并能被广泛接纳的App订餐行为，依然离不开电话直接订购的主要原因^[3]。

在中国，比较成熟的餐饮类App代表之一就是大众点评。大众点评网于2003年4月成立于上海。大众点评是中国领先的城市生活消费平台，也是全球最早建立的独立第三方消费点评网站^[4]。后为了占领移动端的份额，开发了大众点评App，提供商户信息、消费点评及消费优惠等信息服务。截止到2014年第三季度，大众点评月综合浏览量（网站及移动设备）超过80亿，其中移动客户端的浏览量超过80%，移动客户端累计独立用户数超过1.8亿。

国内的一些比较成功的餐饮企业，如辛香汇就已经有了自己的App，通过App可以随时随地，查看体验辛香汇美食，并且可以方便的进行餐位预定，免除排队烦恼，还能随时掌握分店排队叫好情况，免除过号烦恼^[5]。

◎ 订餐系统国外发展现状

在国外，网上订餐的发展比中国早，但智能手机的普及是随着 Android 开源系统的发展而发展的，所以在餐饮 App 的发展上并没有领先中国很多。

说到比较知名的餐饮类 App 就不得不提 Savored。成立于 2010 年的 Savored，通过提供诱人的折扣吸引消费者，目前已经有 1000 多家餐馆与其达成合作。Savored 让用户能够通过价格、位置等条件筛选出最满意的餐馆进行就餐，同时，在这个应用上预订就餐的用户还能得到

15%-40%的全餐折扣，此外，这个应用对消费者是免费的，这些都是它被大量下载的原因。

国外还有一款比较知名的餐饮 App 是 Where To Eat，它可以说是一款“懒汉 App”，它既为你提供传统的找餐馆方法——通过条件搜索选择你所青睐的餐厅，也让你摇摇手机便可找到餐馆——摇动手机，这款应用会为你选择合适的餐馆^[9]。苹果与安卓用户都可以免费使用这款应用。

1.3 选题的先进性和实用性

本系统是针对餐饮企业的开发的移动订餐系统，客户端是基于Android平台开发的，可以方便的完成订餐功能，具有一定的先进性。

使用本系统，在就餐高峰期可以提前预约座位并完成点餐，省去了到餐厅后等待服务员接待以及点餐的时间，同时为用户和商家节约了成本，具有一定的实用性。

1.4 选题的技术难度和工作量

该选题技术难度适中，能够满足研究生论文的研究要求；工作量适中，能够在预定的时间内完成课题研究的内容。

2. 主要参考文献（列出作者、论文名称、期刊名称、出版年月）

[1]冷风. 手机订餐软件卖火了 记者一探究竟. 现代营销. 2014-02-01

[2]赵正. 靠补贴砸市场 订餐 O2O 再现“百团大战”. 中国经济报. 2014-03-10

[3]兰永慧. 对餐饮企业成本管理的思考. 经营管理者. 2014/03

[4]张荣齐; 田文丽 . 餐饮连锁企业 O2O 商业模式研究. 中国市场. 2014/32

[5]李平. 餐饮企业的成本控制. 经营管理者. 2014/10

[6]Roni Elran-Barak;Maya Sztainer;Andrea B. Goldschmidt;Daniel Le Grange . Family Meal Frequency among Children and Adolescents with Eating Disorders Journal of Adolescent Health2013

[7]王舒. 校园智能订餐系统的设计与实现. 电脑知识与技术. 2012/20

[8]唐莉君. 网上自助食堂订餐系统的建模. 电子科技. 2012/11

[9]Dejan sunderic. Microsoft SQL Server 2005 Stored Procedure Programming In T-SQL & .NET, Third Edition. 清华大学出版社. 2008-05

[10]李宁. Android 开发权威指南. 人民邮电出版社. 2013-03

[11]郭宏志. Android 应用开发详解. 电子工业出版社. 2010-06

批注 [雨林木风2]: 要客观分析说明选题的技术难度和工作量在哪里

- [12]叶核亚. Java 程序设计使用教程. 电子工业出版社. 2010. 02
- [13]Maurice Herlihy, Nir Shavit. The Art of Multiprocessor Programming. China Machine Press. 2009-01
- [14]Shari Lawrence Pfleeger, Joanne M. Atlee. Software Engineering Theory and Practice. Higher Education Press. 2009-12
- [15]Thomas H. Cormen Charles E. Leiserson Ronald L. Rivest Clifford Stein. Introduction to Algorithms. China Machine Press. 2004-01

批注 [雨林木风3]: 参考文献列表要规范和统一, 请参考学院论文规范中的参考文献部分。如日期表示都不一致

三、课题内容及具体方案

1. 课题内容

本课题主要是针对苏州订餐困难问题, 而设计的一个移动订餐系统, 其主要包含以下 4 点内容:

- (1) 基于 C/S 架构的移动客户端设计与实现。
- (2) 基于 B/S 架构的网页客户端设计与实现。
- (3) 餐厅查询和订餐。
- (4) 服务器端数据处理。

2. 系统需求分析 (应用软件工程专业描述工具描述)

2.1 概述

移动互联网的发展越来越成熟, 3G 手机用户规模越来越庞大, 这对移动应用的多元化、生活化有了更多的需求。基于位置的服务, 满足了移动用户只需通过手机就能了解到所需服务的位置信息的要求。在餐饮业上, 移动用户除了想了解餐厅的位置信息外, 还希望能提前了解餐厅的菜单是否符合个人喜好, 由此选择适合自己的餐厅和美食, 从而提高用餐质量、节省时间。移动订餐系统就是要提供移动过程中的订餐服务, 为喜爱美食的用户随时随地提供一个搜寻美食, 进而预订美食的一站式服务平台^[7]。同时, 平台也需要网站管理员与餐厅管理员的维护, 为客户提供相应的服务, 保证优质高效的服务质量。因此, 移动订餐系统应在功能上和性能上满足各种用户的要求。

2.2 功能需求

移动订餐系统中包括了普通用户、餐厅管理员和网站管理员三种用户, 根据其不同的角色对系统有不同的需求。

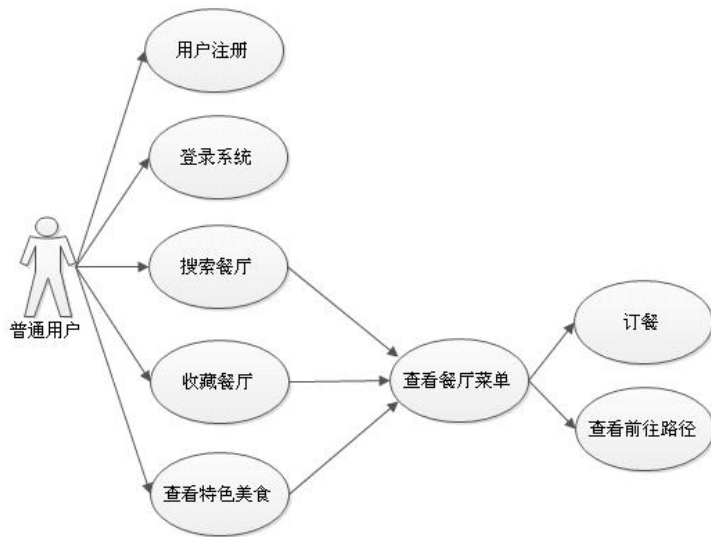


图 2-1 普通用户用例图

批注 [雨林木风4]: 开题报告中的图顺序编号即可。图 1、图 2

移动订餐系统主要服务对象是移动用户^[7]。一般用户在登录系统后需查看网站上推荐的特色美食，自己收藏的餐厅，以及随意搜索需要的餐厅，通过查看餐厅的菜单以及了解其所在位置来决定是否选择该餐厅，并在该餐厅提前订餐。这样用户不但能发掘各地的美食，而且选择适合自己的餐厅，提前订餐能避免因食材不足不能尝试到美食而白走一趟的情况，提前准备保证服务的质量^[8]。普通用户需求用例图如图 2-1 所示。

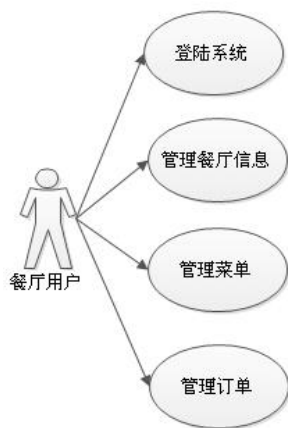


图 2-2 餐厅用户用例图

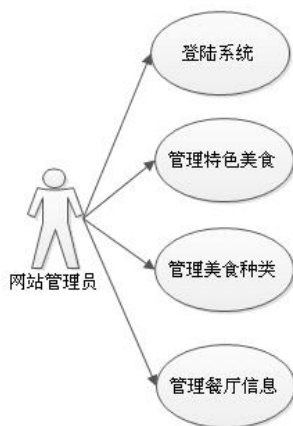


图 2-3 网站管理员用例图

餐厅用户主要为移动用户订餐提供服务。管理员通过管理餐厅信息和餐厅菜单，为一般用户订餐提供参考资料，最后对用户的订单进行管理。餐厅可通过该系统为餐厅起到宣传作用，同时对订单的系统管理，根据预订情况提前准备适量的食材，提高服务效率和质量。餐厅管理员需求用例图如图 2-2 所示。

网站管理员主要是对整个系统进行管理。管理员通过对广告、特色美食、美食种类的管理，使得系统更有条理，更有吸引力和说服力，吸引更多的消费者和商家，提高系统的访问率。网站管理员需求用例图如图 2-3 所示。

本需求着重描述订餐系统设计与实现。此系统主要包括注册模块、登陆模块、周边餐厅搜索、特色菜模块、餐厅菜单模块、订餐模块、路径搜索模块等。

2.3 非功能性需求

- 1) 手机操作系统：系统需要有一定的用户规模，使系统在消费者和商家中都能广泛应用；能提供 GPS 定位设备，实现提供位置信息的功能；操作简便，能很好地访问移动网络，保证系统正常运行^[6]。
- 2) 界面需求：对于一般用户，需要在移动过程中访问系统，界面应该设计友好，结构清晰，操作方便，有良好的用户体验；而对于管理员用户，界面需要能方便访问，对管理项目清晰明了，操作简单。
- 3) 通信网络：根据定位系统要求，能在公共互联网上通信，访问系统服务器，实现数据的传送和接收。

4) 数据存储: 系统需要对所有的数据进行安全的存储, 方便各种用户对数据的操作。在大量用户同时访问时, 保证数据提取的准确性和高效性

3、系统概要设计 (应用软件工程专业描述工具描述)

3.1 目标与概述

本设计将实现在 Android 平台下的移动订餐系统, 系统将集餐厅搜索、餐厅预订和餐厅导航于一身, 为热爱尝试各种美食的用户提供随时随地的移动订餐服务。

本设计需要在设计完成后, 能实现以下功能:

1. 用户可以在 Android 客户端实现用户登录及用户注册功能, 并进入系统 Android 客户端主界面。
2. 在客户端主界面可以实现进入餐厅收藏、查看特色美食和餐厅搜索界面, 并分别实现各个功能。
3. 在客户端实现查看餐厅详情, 并进入餐厅详情界面查看菜单、访问餐厅路径, 最后实现订餐功能。
4. 用户可以通过浏览器登录系统 Web 端主界面, 并实现客户端的相应功能, 网站管理员可进行餐厅信息管理和每日推荐等网站管理, 餐厅管理员可进行上传菜单和处理订单等操作。

另外, 本设计在性能上要达到安全性、可维护性和稳定性三项要求。安全性就是要求对网站管理员、餐厅管理员和一般客户实现对用户操作和数据库访问权限进行管理与控制, 对用户的合法权进行保护。可维护性就是要求在系统开发过程中, 系统文档、指示代码体系遵循软件开发的规范要求。稳定性就是要求系统必须保持性能稳定, 在用户访问时能保证较好的响应速度。

3.2 系统总体设计设计

基于 Android 的移动订餐系统主要是为用户在移动过程中搜罗各种美食, 根据菜单及餐厅位置来选择餐厅, 并作提前订餐, 因此系统要求能在手机操作系统 Android 客户端上访问系统。同时, 为了方便网站管理员经营网站、对网站进行维护和管理, 以及餐厅管理员对餐厅的菜单及订单进行管理, 系统要求能在 Web 浏览器上进行访问, 并实现相应功能。

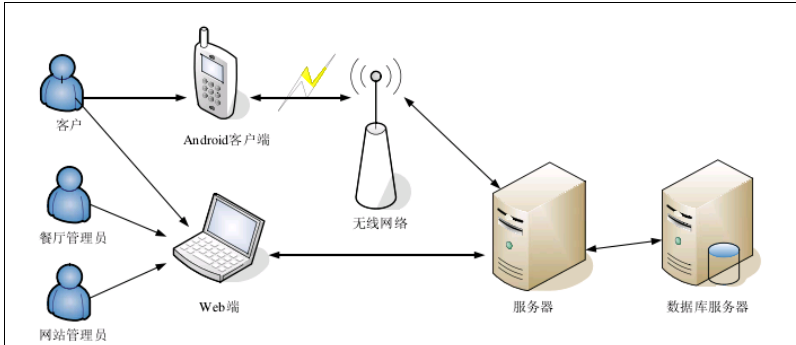


图 3-1 移动订餐系统软件结构

移动订餐系统的总体架构如图 3-1 所示。客户可通过 Android 终端访问移动订餐系统 Android 客户端，通过 Web 浏览器访问系统 Web 客户端；餐厅管理员和网站管理员可通过 Web 浏览器访问移动订餐系统的 Web 管理端，并进行相应的管理和维护。Android 客户端和 Web 端都是通过同一台服务器来操作后台的数据库服务器，实现两端获取的数据的一致性。

3.2.1 Android 客户端开发架构

本系统 Android 端的体系结构如图 3-2 所示，采用 C/S 模式，即客户端/服务器模式。其中，客户端一方面与用户交互，提供良好的用户界面，另一方面与服务器端进行数据交换。服务器端向客户端提供数据下载、数据上传接口以交换数据。服务器端连接数据库，可对数据进行删除、添加、控制等操作。客户端与服务器端通过 Socket 进行数据交换。

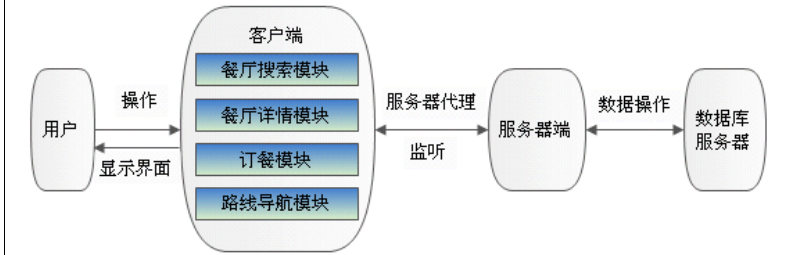


图 3-2 基于 C/S 结构的 Android 客户端架构图

用户通过 Android 设备对 Android 客户端进行操作，视图利用 XML 布局文件进行界面设计，客户端的功能模块采用 Activity 活动类来实现与用户的交互。客户端与服务器端

的沟通要首先通过接口 `getPostExecute` 获取系统信息，在 Tomcat 服务器启动时会客户端监听，然后通过服务器代理线程与客户端通信，最后通过服务器操作到后台数据库。

3.2.2 Web 端开发架构

本系统的 Web 端的体系结构如图 3-3 所示，采用 B/S 模式，即浏览器/服务器模式。在 Browser/Server 的系统中，用户可以通过浏览器向网络上的服务器发出请求。Browser/Server 结构可分为三个单元：表示层（Web 浏览器）、功能层（具有应用程序扩展功能的 Web 服务器）、数据层（数据库服务器）。

为了方便用户在浏览器上进行操作，以及系统各个模块的相对独立，Web 端采用 MVC（Model - View - Controller，模型-视图-控制器）设计模式，将数据显示和数据业务处理分开，使系统在今后的开发中更容易加入新的业务或新类型数据界面，增强系统的可扩展性。

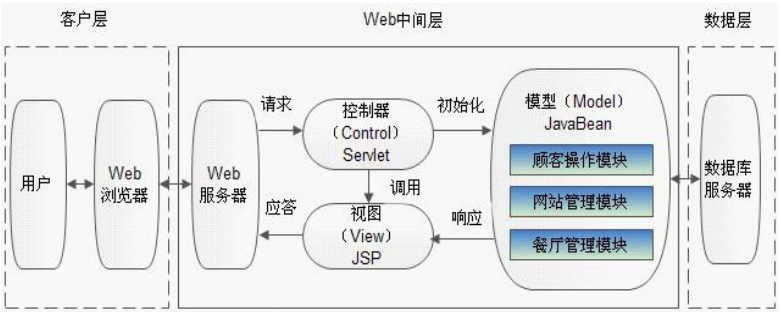


图 3-3 基于 B/S 结构的 Web 端架构图

3.3 功能模块设计

系统的 Android 客户端的主要工作流程如图 3-4 所示。从图中可以看出，用户首先登录系统，或者注册后登录系统后，分别可以进行查看特色美食、查看收藏的餐厅，以及搜索餐厅操作，然后通过点击得出列表中的餐厅进入餐厅的详细信息界面。如果用户对某个餐厅感兴趣，可以进入餐厅详情界面查看菜单及地图位置，当这些符合用户的要求，就可以进入订餐功能模块及餐厅导航功能模块，完成订餐过程。用餐过程中还可给餐厅作评论并上传相关信息。接下来的章节将主要详细介绍餐厅搜索模块、订餐功能模块、餐厅导航模块。

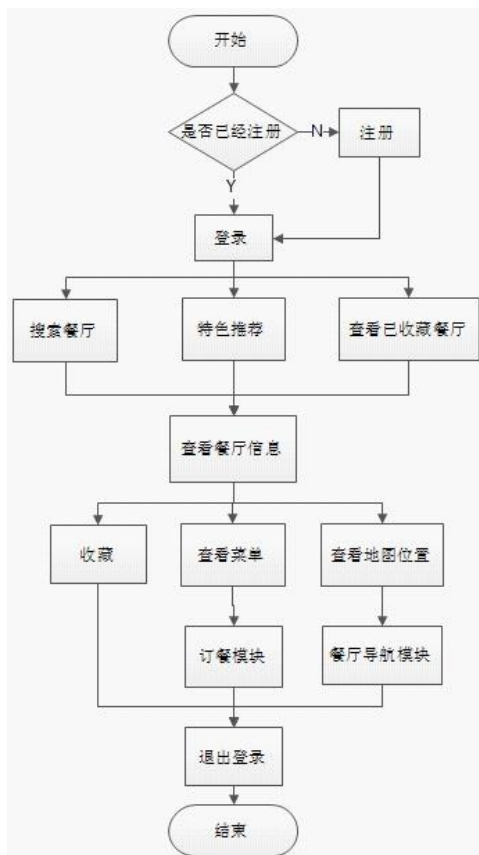


图 3-4 Android 客户端的工作流程图

3.3.2 客户端餐厅搜索模块的设计

餐厅搜索模块可以对数据库中存储的所有餐厅进行搜索，通过关键词、餐厅菜系以及地区信息进行的定性搜索，使用户能方便的搜索到符合自己要求的餐厅。该模块工作流程如图 3-5 所示。

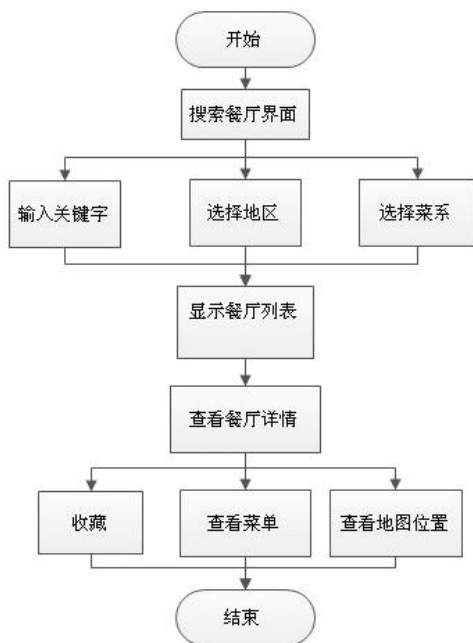


图 3-5 餐厅搜索模块的流程图

从图中可以看到，用户进入到搜索美食界面后可按照关键词、餐厅菜系、地区信息进行搜索，系统将通过访问后台数据库后返回搜索结果。搜索结果界面包括餐厅的名称、菜系和折扣这些简单信息，用户可以通过拖动的方式进入下一页，方便用户查看。点击某个餐厅信息条，可进入餐厅详细信息界面，包括餐厅名称、基本描述、人均消费，还可以查看餐厅地理位置。餐厅详细信息界面还包括“收藏”、“返回”、“餐厅菜单”三个按钮。点击收藏按钮，可以将餐厅作为收藏餐厅，方便日后订餐。点击返回按钮，回到餐厅搜索结果界面。如果需要订餐，那么只需点击餐厅菜单，就可以进入订餐功能模块。

3.3.2 客户端订餐功能模块的设计

订餐功能模块是本设计的核心部分，我们收藏餐厅、查找餐厅的最终目的都是为了品尝美食，提前订餐功能就可以助我们顺利地品尝到美食。该模块工作流程如图 3-6 所示。

从图中可以看出，用户可以通过查看餐厅菜单，包括菜式名称、价格和描述信息，确定是否选择到该餐厅用餐。若符合客户要求，可以点击订餐，打开点菜对界面选择菜品、数量，点菜完成之后可以添加用餐时间和用餐人数以及备注信息。这时可点击提交按钮，就可生成

订单，在个人中心可以查看自己的订单。

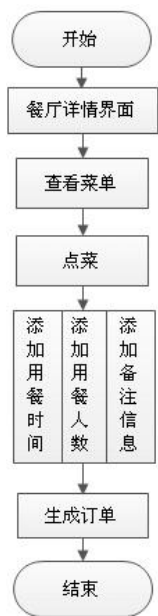


图 3-6 订餐功能模块的流程图

3.3.3 客户端导航功能模块的设计

客户端餐厅导航功能模块是移动订餐系统提供服务的关键。当用户在移动过程中进行订餐，然后需要了解如何到达餐厅的时候，餐厅导航模块为其提供了便利。该模块的工作流程如图 3-7 所示。

从图中可看出，用户在导航界面可以查看用户所在位置以及餐厅位置，并根据两者的位置查询其路径，包括公交线路和地图路径规划，用户可清楚地了解前往餐厅的相关信息和路线情况。

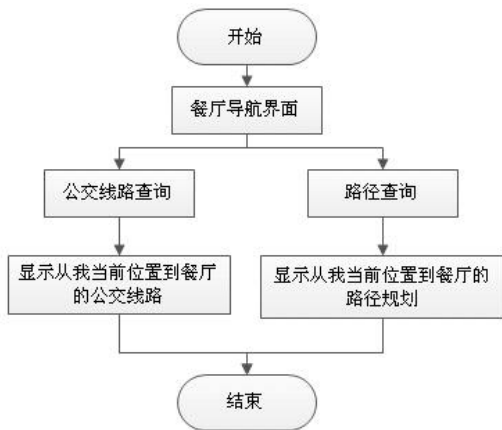


图 3-7 餐厅导航模块的流程图

3.3.4 Web 端管理模块的设计

移动订餐系统可由 Web 端登入进行相关管理，方便网站管理员进行系统维护及餐厅商家的餐厅管理。该模块的工作流程如图 3-8 所示。用户通过 Web 浏览器进入到 Web 管理端主界面，需要进行登录或注册登录后才能进行使用系统。用户类型包括网站管理员、餐厅管理员和普通用户，三者都能进行一般用户类型的操作，同时网站管理员登录后能进入到网站管理模块，餐厅管理员登录后能进入到餐厅管理模块。

接下来本节将主要介绍主界面、餐厅管理模块及网站管理模块的设计，用户操作模块与 Android 客户端的餐厅收藏、餐厅搜索功能基本一致，在此不作详细阐述。

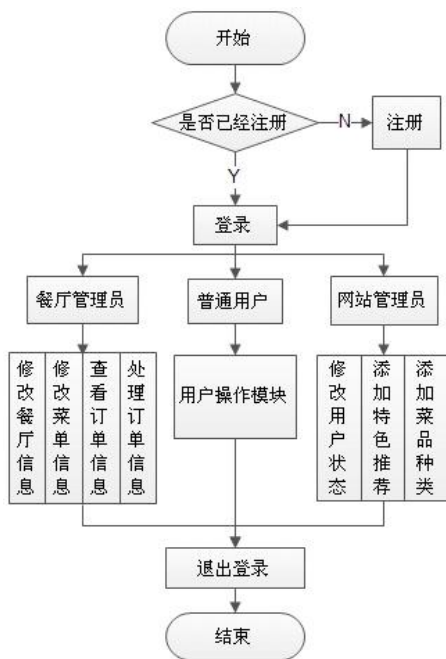


图 3-8 Web 端管理模块的流程图

3.4 数据库设计

对于移动订餐系统，需要存储客户信息、美食信息，地图经纬度、订餐信息等。数据库能实现配置管理系统的数据持久化，是系统的重要组成部分。本文将选用 MySQL 数据库进行后台数据库开发，建立各部分数据表。MySQL 是一个跨平台的开源关系型数据库管理系统，它具有操作简单、性能高、可移植性好、安装时占用的资源少等特点^[9]。因此，本系统的设计采用 MySQL 进行数据库开发。

各个表的功能与结构设计如下：

1. 用户信息表 (tUserInfo)：

序号	字段名	字段说明	类型	是否为空	说明
1	userId	用户 ID	int	N	PK
2	nickName	昵称	varchar(50)	N	

3	userName	姓名	varchar(50)	Y	
4	userPassword	密码	varchar(100)	Y	
5	userSex	性别	int	Y	
6	userImage	头像	int	Y	
7	userPhone	电话	int	Y	
8	userEmail	邮箱	varchar(200)	N	
9	userAddress	联系地址	varchar(500)	Y	
10	tTime	赴约次数	int	N	
11	fTime	爽约次数	int	N	
12	userRegisteTime	注册时间	datetime	N	
13	userStatus	状态	int	N	
补充说明		status: 0-未激活/停用, 1-激活, 2-禁用			

2. 商户信息表（tRestaurantInfo）:

序号	字段名	字段说明	类型	是否为空	说明
1	restaurantId	商户 ID	int	N	PK
2	restaurantName	商户名称	varchar(100)	N	
3	restaurantCategory	商户类型	int	N	
4	restaurantBoss	负责人	Int	N	
5	restaurantEmail	商户邮箱	Varchar(200)	N	
6	restaurantPassword	密码	int	N	
7	restaurantPhone	联系号码	int	N	
8	restaurantAddress	商户联系地址	varchar(500)	N	
9	restaurantRoute	行车路线	varchar(500)	Y	
10	restaurantDescription	餐厅介绍	varchar(500)	Y	
11	expenditure	人均消费	int	Y	
12	coordinateX	地址坐标 X	float(4)	Y	

13	coordinateY	地址坐标 Y	float(4)	Y	
14	locationId	所属区域	int	N	FK
15	takeawayRange	送餐半径	int	N	
16	defaultPrice	起送金额	Int	N	
17	freight	运费	float	N	
18	freeFreightPrice	免运费金额	float	N	
19	bookDate	预订天数	int	N	
20	aveFlavorComment	平均口味	float	N	
21	aveEnvironmentComment	平均环境	float	N	
22	aveServiceComment	平均服务	float	N	
23	avePerformanceComment	平均性价比	float	N	
24	rStartTime	营业起始时间	dateTime	N	
25	rEndTime	营业结束时间	dateTime	N	
26	rRegisteTime	加盟时间	dateTime	Y	
27	rStatus	账号状态	Int	Y	
补充说明		restaurantCategory: 1-餐厅预订, 2-外卖预订, 3-餐厅、外卖预订			

3. 菜肴类别表 (tDishesCategory):

序号	字段名	字段说明	类型	是否为空	说明
1	categoryId	类别编号	int	N	PK
2	categoryName	类表名称	Varchar(100)	N	
3	categoryStatus	类别状态	int	N	
补充说明		categoryStatus: 1-审核未通过, 2-审核通过			

4. 菜肴信息表 (tDishesInfo):

序号	字段名	字段说明	类型	是否为空	说明
1	dishesId	菜肴 ID	int	N	PK
2	restaurantId	商户 ID	int	N	FK

3	dishesName	菜肴名称	varchar(100)	N	
4	dishesImage	菜肴图片	varchar(100)	Y	
5	categoryId	菜肴类别	int	N	FK
6	dishesDescription	菜肴描述	varchar(1000)	N	
7	answerStartTime	菜肴价格	dateTime	N	
8	answerEndTime	是否推荐	dateTime	N	
9	clickTime	被点次数	int	N	
10	dishesStatus	菜肴状态	int	N	
补充说明		DishesStatus: 0-删除, 1-未删除			

5. 特色推荐表 (tDishesRecommend):

序号	字段名	字段说明	类型	是否为空	说明
1	recommend	推荐 ID	int	N	PK
2	dishesId	菜肴 ID	int	N	FK
3	rStartTime	开始时间	dateTime	N	
4	rEndTime	结束时间	dateTime	N	
5	managersId	推荐管理员 ID	int	N	FK
6	recommendTime	推荐时间	dateTime	N	
补充说明					

6. 商业定位表 (tBusiness):

序号	字段名	字段说明	类型	是否为空	说明
1	businessId	商业定位表 ID	int	N	PK
2	restaurantId	商户 ID	int	N	FK
3	businessCategoryId	商业定位类别 ID	int	N	FK
补充说明					

7. 菜系类别表 (tCuisineCategory):

序号	字段名	字段说明	类型	是否为空	说明
----	-----	------	----	------	----

1	cuisineCategoryId	菜系类别 ID	int	N	PK
2	cuisineCategoryName	菜系名称	int	N	
3	upcuisineCategoryId	上级菜系 ID	int	N	
补充说明					

8. 商户菜系表 (tCuisine):

序号	字段名	字段说明	类型	是否为空	说明
1	cuisineId	商户菜系表 ID	int	N	PK
2	restaurantId	商户 ID	int	N	FK
3	cuisineCategoryId	菜系类别 ID	int	N	FK
补充说明					

9. 订单信息表 (tOrderFormInfo):

序号	字段名	字段说明	类型	是否为空	说明
1	OrderFormId	订单 ID	int	N	PK
2	restaurantId	商户 ID	int	N	FK
3	userId	用户 ID	int	N	FK
4	orderFormCategory	订单类别	int	N	
5	diningTime	就餐时间	dateTime	N	
6	diningNum	就餐人数	int	N	
7	orderFormTime	下单时间	dateTime	N	
8	bookName	预订者姓名	varchar(100)	N	
9	bookSex	预订者性别	int	N	
10	bookPhone	手机号	varchar(20)	N	
11	orderFormDescription	备注	Varchar(1000)	Y	
12	isComement	是否评价	int	N	
13	orderFormStatus	订单状态	int	N	
补充说明		OrderFormCategory: 1-餐厅预订, 2-外卖预订 bookSex: 0-男, 1-女			

	isComement: 0-否, 1-是
--	----------------------

10 订单菜肴信息表 (tOrderFormDishesInfo):

序号	字段名	字段说明	类型	是否为空	说明
1	OrderFormId	订单 ID	int	N	PK, FK
2	dishesId	菜肴 ID	int	N	FK
3	dishesNum	菜肴份数	int	N	
4	dishesPrice	菜肴价格	float	N	
补充说明					

11. 商户评价表 (tRestaurantComment):

序号	字段名	字段说明	类型	是否为空	说明
1	restaurantCommentId	商户评价表 ID	int	N	PK
2	restaurantId	商户 ID	int	N	FK
3	userId	用户 ID	int	N	FK
4	flavorComment	口味	int	N	
5	environmentComment	环境	int	N	
6	serviceComment	服务	int	N	
7	performanceComment	性价比	int	N	
8	userAdvices	食客评语	varchar(500)	N	
9	commentTime	点评时间	dateTime	N	
补充说明					

12. 收藏信息表 (tCollectinInfo):

序号	字段名	字段说明	类型	是否为空	说明
1	userId	用户 ID	int	N	PK, FK
2	restaurantId	商户 ID	int	N	PK, FK
补充说明					

13. 管理员信息表 (tManagersInfo):

序号	字段名	字段说明	类型	是否为空	说明
1	managersId	管理员 ID	int	N	PK
2	managersName	用户名	varchar(100)	Y	
3	managerPassword	密码	varchar(100)	Y	
补充说明					

14. 优惠信息表 (tDiscountInfo):

序号	字段名	字段说明	类型	是否为空	说明
1	discountId	优惠信息 ID	int	N	PK
2	restaurantId	商户 ID	int	Y	FK
3	userId	优惠类型	int	Y	
4	discount	折扣	varchar(50)	N	
5	discountDiscription	优惠描述	Varchar(400)	N	
6	dStartTime	起始时间	DateTime	N	
7	dEndTime	结束时间	DateTime	N	
补充说明					

4. 拟采用的设计方法、环境等

在移动客户端采用现有的相对成熟的手机系统开发框架，基于 HTTP 协议，实现客户端与服务端通信，并通过百度地图 SDK 实现路线规划和定位功能。

在网页客户端采用 SSH2 框架，并通过 Ajax 技术实现前台和后台数据逻辑处理及通信。

在服务端，通过 Servlet 小程序，实现对移动客户端 HTTP 请求的结果返回，并采用数据库连接池对数据库连接进行维护，健壮服务器的性能，保证客户端的一些非功能性要求。

开发环境为 Eclipse、MyEclipse，Web 容器为 Tomcat server，数据库为 Mysql。

5. 技术难度及特色分析

按照移动互联网应用系统的开发模式，并不需要特别炫酷的技术，其采用相对成熟的开发技术，保证系统开发的速度和稳定。而面对 iOS/Android/Windows Phone 所带来的新的语

言学习成本，开发框架的学习成本以及开发模式的开发成本，都成为了企业业务系统开发人员的最大障碍之一。

多平台带来的人力成本的开销。因为多移动平台的存在，因此原先只要使用一种技术，统一在 Web 里面实现的局面被打破了。企业需要的是懂 iOS/Android/Windows Phone 开发的三类人员，甚至需要把同一个业务需求做三遍。这对企业来说是非常痛苦的一件事情，而同时因为开发人员的短缺，同时会导致企业的用户成本直接增加。

本系统拥有很多特色

1. 本系统采用了移动云服务，计算资源以及存储容量无上限。
2. 本系统作为移动互联网应用系统，其具有随时随用的特性。
3. 本系统作为 Android 平台的系统，具有方便使用，用户广的特性。
4. 支持导航系统。
5. 支持定位功能，用户可以很方便的查看周边餐厅位置信息。
6. 服务器处理、响应速度快，且相当稳定。

6、本人主要工作描述

移动订餐系统是一个十分复杂的基于移动云服务的移动互联网应用系统，它包括两个前台客户端和一个后台管理。在服务端，对于客户端的请求，Server 会启动并调用 Servlet，Servlet 根据客户端的请求生成响应内容并将其传给服务器，服务器会将响应返回客户端。我的工作量主要是负责客户端框架的搭建、以及一些 UI 组件的编写、客户端和服务端通信接口的编写等。

四、工作进度的大致安排

批注 [雨林木风5]: 标题不要放在一页的最后

应包括文献调研，工程设计，新工艺、新材料、新设备、新产品的研制和调试，实验操作，实验数据的分析处理，撰写论文等。

时间	工作	阶段成果
2014.5 - 2014.7	完成对移动订餐导系统的市场调研，分析项目的可行性。	可行性分析报告，市场调研，开题报告
2014.7 - 2014.9	完成前期课题调研，研究分析相关技术，并掌握项目总体框架的设计	需求分析报告
2014.9 - 2014.11	掌握项目各模块具体的原理以及自己相关工作的重点和难点，熟悉系统架构的每个细节	概要设计报告
2014.11 - 2015.1	根据设计文档完成自己相关模块的工作的详细设计，部署好开发环境，完成代码框架的编写	详细设计报告
2015.1 - 2015.2	完成项目编码与测试工作	项目代码
2015.2 - 2015.3	整理相关文档，撰写毕业论文，进行毕业论文答辩	毕业论文初稿
2015.3 - 2015.4	修改和完善毕业论文	毕业论文定稿

预期成果

- (1) 完成基于 Android 的移动订餐系统的设计与实现
- (2) 撰写工程硕士学位论文

二、选题依据

1. 阐述该选题的研究意义，或工程设计的价值和意义，国内外概况和发展趋势，选题的先进性和实用性，技术难度及工作量。

1.1 选题的研究意义

在全自动化的生产线控制系统中，有着非常复杂的控制流程，而在传统的系统设计中，这些控制流程和业务逻辑是混合在一起的，即使采用了组件式的设计，仍然需要“胶水”代码^[1]将业务逻辑结合到一起运行。这样的方式使得系统的控制流程分散在系统的各个部分，为系统的后续维护和扩展带来了很大的困难。而企业为了应对不断变化的市场环境，经常需要对控制流程做出调整，对系统的灵活性有着很高的要求，因此如何有效的分离系统的控制流程和业务逻辑，实现流程的可视化和标准化管理，增强系统的灵活性和可维护性，是迫切需要解决的问题。

工作流技术作为一种实现流程自动化的技术在企业办公自动化，系统集成以及应用内部流程驱动方面都有着广泛的应用^[2]。它有效的实现了控制流程和业务逻辑的分离，可以很好的应对流程的变化。在应用系统的设计中，引入工作流引擎作为系统的驱动核心，按照定义的流程文件，对业务逻辑组件进行编排执行，从而实现控制流程，可以大大简化系统结构，同时实现了流程的灵活管理，增强了系统的可扩展性和可维护性。

工作流技术从产生至今，出现了很多的规范和产品，但是主要集中在企业办公自动化领域，作为应用系统驱动内核的工作流引擎产品很少，而且一般都是嵌入在别的应用系统中的，通用性较差。近年来随着SOA（Service Oriented Architecture）技术^[3]的发展，基于BPEL(Business Process Execution Language)^[4,5]的工作流技术和产品因其良好的可执行性逐渐成为了主流。但是其主要的目的是为了系统间集成，所以引入了Web Service技术^[3]，几乎所有的BPEL工作流引擎产品为了与标准兼容，都实现了Web Service。Web Service技术对于系统间的服务调用是合适的，但对于应用内部的业务逻辑组件的调用，则显得多余，而且Web Service的开销大，对于生产线控制这样对实时性要求很高的系统是不合适的。所以直接利用现有的产品比较困难，而利用BPEL规范定义良好的体系结构，在考虑具体应用环境的基础上进行适当剪裁，设计一个轻量级的工作流引擎，作为自动化生产线控制系统的驱动核心，对系统结构进行改造，从而实现控制流程和业务逻辑的分离，提高系统的可维护性和可扩展性是一个值得研究的课题。

1.2 国内外概况和发展趋势

1.2.1 工作流的定义

批注 [雨林木风1]: 工作流引擎国内外技术现状及发展趋势调研不够充分，没有聚焦。

workflow 技术经过了几十年的发展，但是到目前为止还缺乏一个统一的定义，不同的研究者和 workflow 产品供应商从不同的角度给出了 workflow 的定义。workflow 技术的标准化组织 workflow 管理联盟 (WfMC) 给出的 workflow 的定义^[6]为：workflow 就是一类能够完全或者部分自动执行的经营过程，它根据一系列过程规则，文档、信息或人物能够在不同的执行者之间进行传递与执行。

1.2.2 workflow 技术的起源和发展历史

workflow 技术发端于 1970 年代中期办公自动化领域的研究工作。1970 年代 ZISMAN 在他的原型系统 SCOOP 中描述了使用信息系统来支持组织流程的思想，该系统采用 Petri-nets 来表达流程。1975~1985 年间对于办公自动化的研究成爆发型的增长，奠定了 workflow 技术在管理流程领域工业化应用的基础。1980 年代末兴起了对群件技术和 workflow 技术的研究，1990 年代伴随着全面质量管理，流程改进，业务流程革新，业务流程重组等基于流程的最佳实践的推广和实施，workflow 技术的发展也迎来了繁荣发展的时期^[2,7]。

1993 年 8 月，workflow 技术标准化的工业组织-workflow 管理联盟 (WfMC) 成立。1994 年，workflow 管理联盟发布了用于 workflow 管理系统之间互操作的工作流参考模型，并相继制定了一系列工业标准。关于 workflow 技术的学术研究也十分活跃，许多原型系统在实验室里开发出来，人们从 workflow 模型、体系结构、事务、适应性、异常、安全、语言、形式化、正确性验证、资源管理、开发过程等各方面对 workflow 技术进行探讨。

进入 2000 年以后，随着 Web 服务技术的兴起，多个标准化组织制定了各自和 workflow 技术相关的 Web 服务标准，如 XLANG，WSFL 等。2002 年 8 月，IBM，Microsoft 为等企业联合提交并发布了 BPEL4WS 规范，2007 年 4 月 11 日，OASIS 发布了 WS-BPEL2.0 标准。各类支持 BPEL 的产品也如雨后天春笋般出现，对于 workflow 技术的研究和应用进入了一个新的热潮时期。

尽管 workflow 技术的研究和应用取得了进步，但理论基础的研究还远远不够。现有的 workflow 管理系统从功能、可靠性、健壮性上与数据库管理系统仍无法相提并论^[7]。

1.2.3 workflow 技术的应用

目前的 workflow 技术主要应用于三个领域：办公自动化，系统集成，workflow 驱动的应用软件^[2]。

办公自动化领域是 workflow 技术的发源地，应用非常广泛，目前几乎所有的办公自动化产品都支持 workflow，其他的业务系统如 ERP 等大多也包含 workflow 管理的功能。其应用的目的是为了实 现业务流程的自动化，例如企业的订货处理，财务审批，政府的行政审批，公文流转等。流程的自动化大大提高了业务处理的效率，降低了业务成本和风险，为应用领域带来了巨大的收益。该领域的产品往往都是基于特定的应用需求进行定制开发的，因而是和业务系统紧密结合在一起的，缺乏通用性。

workflow 技术应用于系统集成领域是随着信息技术的快速发展和应用而兴起的。传统的企业组织架构都是基于职能部门的，随着信息技术在企业的广泛应用，各个职能部门往往都使用一些信息系统用于业务的处理，这样在企业中往往同时存在很多为各种目的而使用的系统，各系统由于投入使用的时间不同，厂商不同，技术平台不同等各种障碍难以进行集成。然而进入新世纪以来，随着市场竞争的加剧，企业为了提高竞争力需要不断的优化自身的业务流程，进行业务流程重组。新的业务流程需要打通部门间甚至关联业务伙伴企业之间的沟通的壁垒，同时也需要各部门，各业务伙伴企业的系统间能够协同工作。这种系统间集成的需求推动了系统集成技术的发展，WEB 服务技术的出现为跨系统，跨平台的系统集成提供了一种很好的解决方案，也催生了基于WEB 服务的工作流技术，基于BPEL 工作流定义语言的工作流技术得到了各大主流厂商的支持，大部分的产品都宣称兼容BPEL 标准。

workflow 技术用于驱动软件流程并没有引起足够的关注，因为 workflow 技术从产生到现在，主要服务于业务流程处理，针对开发人员的工作流技术应用并不多见^[1,8]。目前 workflow 驱动的应用软件主要存在于自动化程度较高的应用领域，如自动化生产流水线控制，自动任务处理等领域。其应用领域具有一些特殊性，即应用程序的运行根据事先定义好的策略进行自动运行，具有自适应性，运行的过程中往往不需要人的干预，只有在发生异常情况时才需要人为的处理。这类产品往往与特定的领域相关联，通用性不强，也往往被作为自动控制软件而不划入 workflow 技术产品的范畴。还有一些业务规则引擎其实也包含 workflow 的功能，例如ILOG JRules中就有Flow控制的功能。目前这一领域的研究和应用还不成熟，缺乏领域独立的产品，但是随着 workflow 技术的发展，特别是可执行的工作流技术的发展，将 workflow 技术引入应用程序的结构设计中，进而简化应用系统设计，提供系统的可理解性，灵活性和可维护性，具有良好的应用前景，本课题正是在该领域中的一个研究尝试。

1.2.4 流程定义语言规范

workflow 技术相关的标准中，最重要的就是流程定义语言规范，因为 workflow 是基于流程定义语言定义的规则进行运行的。目前的流程定义语言规范很多，但总体上可以分类两类：基于标准XML 文档的规范和基于WEB 服务的技术规范^[9]。

基于标准XML 文档的规范，包括：

1) XPDL^[10] (Xml Process Definition Language)

1994年11月，WfMC 发布了 workflow 管理系统的参考模型。参考模型提出了五类接口，有关过程模型的定义则构成了接口一的核心内容。接口一早期的标准为WPD L (Workflow Process Definition Language)，后来这一接口的规范变更为XPDL。XPDL 是至今 workflow 领域最为重要的一

个标准，目前最新版本为2.2。大多数 workflow 引擎是依据该标准设计开发的。

2) BPML^[11] (Business Process Modeling Language)

BPML是BPMI (Business Process Management Initiative) 组织发布的规范。WfMC和BPMI在2002年6月26日宣布将合作制定业务流程和工作流标准，即采用BPML来描述工作流过程，同时采用XPDL所定义的工作流模型。BPML规范为表达业务流程和支持实体提供一个抽象模型。BPML为表达抽象和执行流程定义了一种正式模型，该模型代表了企业业务流程的面貌，包含了不断变化的复杂行为，事务和数据管理，合作，异常捕获，操作语义。BPML为了能够持久化和通过异种系统进行定义交换以及使用建模工具，提供了 XML Schema 形式的语法。现在，曾提出BPML语言的BPMI已经放弃对其的支持，转而推广BPEL。

3) WfMF^[12] (Workflow Management Facility)

在WfMC所定义的一系列规范基础上，OMG (Object Management Group) 联合这些规范发布了 Workflow Management Facility 规范，该规范定义了如何将工作流向CORBA 转换。

基于 Web 服务技术的规范，包括：

1) WSCI^[13] (Web Services Choreography Interface)

2002年6月26日BEA、Intalio、SAP和Sun在美国发布了基于XML的Web服务协作接口WSCI。WSCI描述了在特殊流程中通过Web服务实现消息流的交流，并描述了集合性信息在互动的Web服务间的交流，提出了一种涉及到多种Web服务的复杂流程的观点。WSCI的关键优势之一在于，它通过描述 Web 服务如何在大型的、全面的业务流程中应用，从而在业务流程管理与Web服务之间架起了桥梁。这些业务流程可以只是一个公司内的，也可以是跨越多个公司的。后来又发展了几个新的规范，如WSFL (IBM)，XLang (MS)。如今WSCI的几个厂商如BEA/SAP/SUN等均已转到BPEL，WSCI基本上没有了发展的空间。

2) ebXML^[14] (Electronic Business XML)

ebXML是一个规范集，这些规范共同实现了模块化电子商务框架。 ebXML的构想是实现一个全球电子市场，其中，不同规模和不同地区的企业可以通过交换基于XML 的消息来合作和进行商业活动。ebXML是一项倡议，其参与者与认可者包括几百家大公司和团体。许多标准团体也参与其中。ebXML主要应用于电子商务领域。

3) BPEL (Business Process Execution Language for Web Services)

2002年8月9日，Microsoft、BEA、IBM、SAP & Siebel联合提交发布了BPEL规范，整合了XLANG，WSFL，BPML。此规范描述如何处理输入的消息，它不是一个关于业务流程规格化定义的规范，简单的说，可以将它看作XML形式的编程语言，提供将WSDL-Services组合成控制流的能力。2007年4月11日，OASIS推出了WS-BPEL2.0。

综上所述,目前较活跃和通用的流程定义语言规范主要是XPDL和BPEL,XPDL偏重于对业务人员的支持,关注业务流程的表达;而BPEL则更偏重于对开发人员的支持,关注流程的自动化执行,因其可执行性更好,更适合用于流程引擎的开发,本课题选用BPEL作为流程定义语言。

1.3 选题的先进性和实用性

BPEL是广泛使用的工作流建模语言,得到各大主流厂商的大力支持,基于BPEL的工作流引擎的商业和开源产品都得到了广泛的应用,经过了实践的检验,是值得信赖的。该选题基于BPEL规范,但是根据实际的情况对规范进行了适当的剪裁,在引擎的设计中去掉了Web Service部分,使得引擎变得更“轻量级”,消除了可能因Web Service带来的性能和执行速度的问题。同时系统采用开放式的结构设计,可以通过接口组件的方式对Web Service/MOM等多种集成技术提供支持,具有一定的先进性。

此外,该选题所设计的工作流引擎,具有应用领域独立性,可以很容易的集成到其他应用系统中,作为应用的驱动引擎,优化系统结构,实现控制流程和业务逻辑的解耦,增强系统的灵活性和可维护性,提高应用的生命周期,所以具有很强的实用性。

1.4 选题的技术难度和工作量

该选题技术难度适中,能够满足研究生论文的研究要求;工作量适中,能够在预定的时间内完成课题研究的内容。

批注 [雨林木风2]:要客观分析说明选题的技术难度和工作量在哪里。

2. 主要参考文献（列出作者、论文名称、期刊名称、出版年月）

- [1] Andrew C. Oliver. Fire up a Workflow engine to improve software development. Available: <http://www.infoworld.com/article/2615751/development-tools/fire-up-a-workflow-engine-to-improve-software-development.html> [2014, September 05]
- [2] Jörg Becker and Michael zur Muehlen. Workflow Application Architectures: Classification and Characteristics of Workflow-based Information Systems. Workflow Handbook 2002: 39~50
- [3] 王紫瑶. 2008. SOA 核心技术及应用. 北京: 机械工业出版社
- [4] BEA,IBM,Microsoft etc. Business Process Execution Language for Web Services Version 1.1. Available: <http://msdn.microsoft.com/en-us/library/aa479358.aspx> [2014, September 21]
- [5] OASIS Standard. Web Services Business Process Execution Language Version 2.0. 2007
- [6] 罗海滨,范玉顺,吴澄. 工作流技术综述. 软件学报,2000,11(7):899~907
- [7] 范玉顺. 2001. 工作流管理技术基础. 北京: 清华大学出版社
- [8] Dragos-Anton Manolescu. Micro-Workflow: A Workflow Architecture Supporting Compositional Object-Oriented Software Development: [PhD Thesis]. Illinois: University of Illinois at Urbana-Champaign, 2001
- [9] 孟彦敏. 基于WF工作流技术的应用研究:[硕士学位论文]. 南京:南京邮电大学, 2010
- [10] WfMC. XML Process Definition Language Version 2.2. Available: [http://www.xpdl.org/standards/xpdl-2.2/XPDL%202.2%20\(2012-08-30\).pdf](http://www.xpdl.org/standards/xpdl-2.2/XPDL%202.2%20(2012-08-30).pdf) [2014,October 18]
- [11] BPMI. Business Process Modeling Language Version 1.0. Available: <http://www.omg.org/bpmn/Documents/BPML-2003.pdf> [2014,October 18]
- [12] OMG. Workflow Management Facility Specification V1.2. Available: <http://doc.omg.org/for mal/2000-05-02.pdf> [2014,October 18]
- [13] W3C. Web Service Choreography Interface (WSCI) 1.0. Available: <http://www.w3.org/TR/wsci/> [2014,October 18]
- [14] OASIS. ebXML Business Process Specification Schema Technical Specification v2.0. Available: <http://docs.oasis-open.org/ebxml-bp/2.0.4/OS/spec/ebxmlbp-v2.0.4-Spec-os-en-html/ebxmlbp-v2.0.4-Spec-os-en.htm> [2014,October 18]
- [15] W.H.M. van der Aalst. 2003. Workflow Patterns. Netherlands: Kluwer Academic Publishers
- [16] 彭炜. 基于 BPEL 的工作流引擎的研究与设计:[硕士学位论文]. 长沙:国防科学技术大学, 2007
- [17] 佟伟. 基于 SCA 的轻量级 BPEL 引擎的设计与实现:[硕士学位论文]. 北京:北京工业大学, 2008
- [18] 林川捷. 基于 BPEL4WS 规范的工作流引擎的研究与实现:[硕士学位论文]. 北京:北京邮电大学, 2008
- [20] 杨春. 基于 BPEL4WS 规范的工作流管理系统的研究与实现:[硕士学位论文]. 广州:华南理工大学, 2005
- [21] 杨明顺. 一种轻型工作流引擎的设计与实现. 西安理工大学学报, 2013, 29(1)
- [23] 计文柯. 2012. Spring 技术内幕 (第2版). 北京: 机械工业出版社

批注 [雨林木风3]: 参考文献列表要规范和统一。如日期位置都不一致, 请参考学院论文规范。正文中引用要标注。

三、课题内容及具体方案

1. 课题内容

- (1) 调查和分析 workflow 引擎在国内外的应用和发展状况。
- (2) 分析 BPEL workflow 引擎的上下文环境，展开需求分析。
- (3) 研究 BPEL 规范，并根据需求对其进行剪裁。
- (4) 研究 workflow 模式，设计 BPEL workflow 引擎内核。
- (5) 用 Spring 实现和验证 BPEL workflow 引擎。

2. 拟采用的设计方法

本方案拟采用面向对象的设计方法，同时使用了大量的设计模式，整体设计呈现开放式的结构，便于功能的扩展和持续的改进。

3. 技术路线

本方案首先对引入 workflow 引擎后的应用系统结构进行分析，理解 workflow 引擎的上下文环境，展开需求分析，然后设计引擎的核心结构，最后在 Spring 框架下进行实现和验证。

批注 [雨林木风4]: 放在后面

3.1-2 需求分析

3.1.1-2.1 workflow 引擎应用上下文分析

如图 3-1 所示，自动化生产线控制系统是由多个功能相对独立的系统构成，这些系统虽然功能和使用目的不同，但是都具有相似的结构，而且都是基于设备的事件驱动的，所以一般采用分布式的结构，通过企业服务总线集成起来。

单个应用系统的内部结构相似，都采用领域建模、三层结构设计，典型结构如图 3-2 所示。这样的系统结构虽然属于比较规范的，但是因为自动化生产控制关联的控制流程非常复杂，导致控制层代码臃肿，缺乏灵活性，控制流程变更时需要变更程序，给后期的维护和扩展带来了很大的困难。而且这些系统往往缺乏完善的文档，随着开发和维护人员的流动，到后期可能会出现不敢变更的情况，因为复杂的代码修改风险很高，稍有不慎就可能造成问题，给公司带来损失。

引入 workflow 引擎之后的应用系统结构如图 3-3 所示。引入 workflow 引擎之后，对业务的编制都是通过 workflow 定义文件进行定义的，而定义的工作流文件通过 workflow 引擎解释执行，调

批注 [雨林木风5]: 开题报告中的图顺序编号即可。图 1、图 2

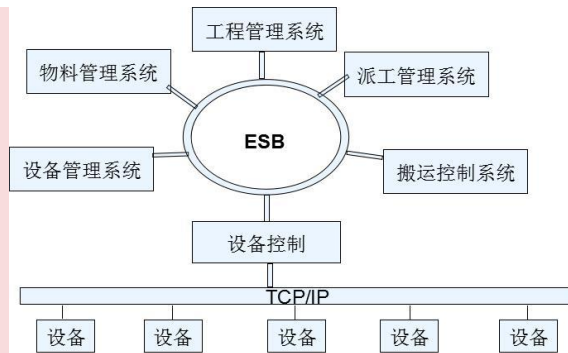


图3-1 自动化生产线控制系统架构

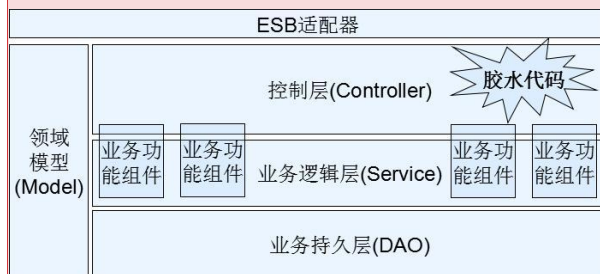


图3-2 应用系统的结构

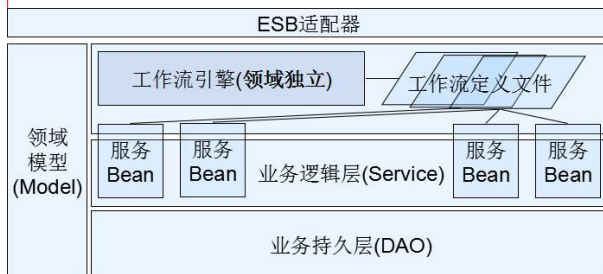


图3-3 引入工作流引擎后的应用系统的结构

用业务领域中的业务逻辑组件完成实际的功能。这样工作流引擎是领域独立的，所有的系统都可以嵌入同样的工作流引擎，系统结构的统一性将节省开发和维护的成本。同时，工作流引擎将系统的结构更彻底地解耦，控制流程变更时只需要修改对应的工作流定义文件，业务逻辑组件扩展时工作流引擎不需要任何变动。这增强了系统灵活性，有利于延长系统的生命周期，节约投资。同时，工作流定义文件将控制流程逻辑统一进行管理，其本身就是最好的说明文档，为系统的后期维护和扩展提供了便利。

3.1.22.2 工作流引擎用例分析

由于本课题只涉及工作流引擎的设计和实现，故未考虑流程定义的可视化实现，工作流定义采用 xml 文件的方式(下一节中进行详细说明)。同时工作流引擎以组件的方式嵌入到应用系统中，实际与工作流引擎交互的关联者有应用容器，消息分配器(ESB Adapter)，应用业务组件，监控组件，对应的用例如下：

用例 UC-01 应用容器用例（如图 3-4），由于引擎需要以组件的方式运行在应用系统的容器中，所以容器需要对引擎进行初始化。引擎初始化是指容器创建引擎实例，并设置相应的参数，同时引擎在初始化时还需要解析和注册定义好的 BPEL 流程文件。

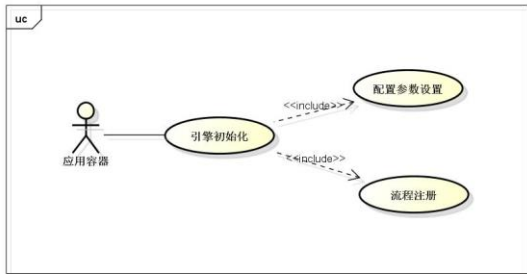


图3-4 用例UC-01 应用容器用例图

用例 UC-02 消息分配器用例（如图 3-5），消息分配器是应用系统容器中处理 ESB 消息的接口组件，它接收来自 ESB 的消息，并将其 Mapping 到对应的流程，然后调用引擎执行流程。流程执行包括新的流程执行以及唤醒睡眠的流程继续执行。

用例 UC-03 业务功能组件用例（如图 3-6），在流程的执行中，需要调用应用的业务功能组件来完成相应的业务处理。同时，一个流程也可以调用另一个流程。

用例 UC-04 监控组件用例（如图 3-7），工作流引擎需要提供流程监控接口以及事件接口，以便于应用对工作流执行情况进行监控以及满足调试，追踪，模拟的功能要求。

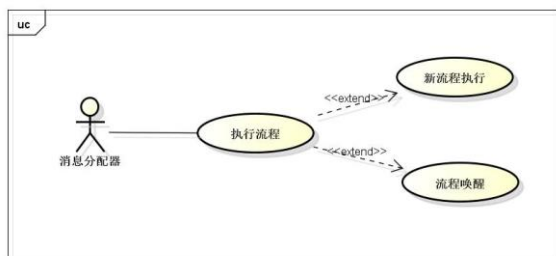


图3-5 用例UC-02 消息分配器用例图

powered by Astah

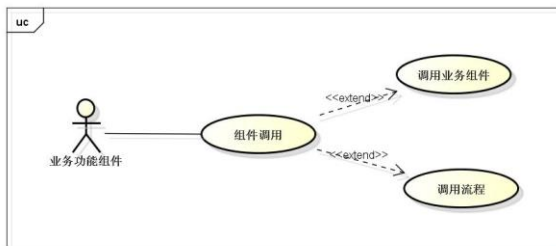


图3-6 用例UC-03 业务功能组件用例图

powered by Astah

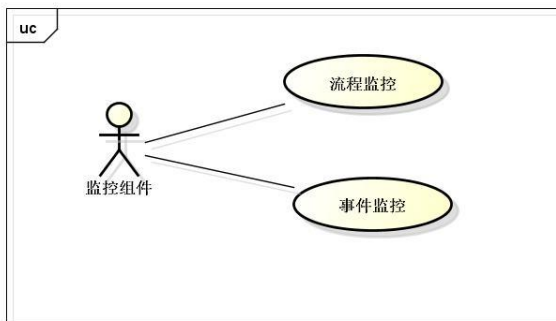


图3-7 用例UC-04 监控组件用例图

powered by Astah

3.1.32.3 流程定义文件规范

流程定义文件规范是指流程定义时需要遵循的语法和约束。流程定义文件是应用系统和

工作流引擎的接口，应用将实际的控制流程抽象定义为流程文件，工作流引擎则按照流程定义文件规范对流程文件进行解析和执行。本课题在 BPEL 规范的基础上，结合应用需要支持的流程的实际需求，制定了流程定义文件的规范，以 DTD 文件的形式产出（见附录 A）。

在 BPEL 流程定义中，流程是由一系列的活动构成的，主要的活动说明如下：

Process，定义一个流程，流程文件必须以“Process 名.bpel”的格式命名。

PartnerLinks，可调用的组件的定义，类型需要支持对应用业务组件以及流程的调用。

Variables，流程执行过程中使用的变量的定义，定义变量的名称、类型、值等。

Expression，流程执行过程中需要执行的表达式的定义，用 Xpath Function 定义。

FaultHandler，异常处理流程定义。

Catch/Catch ALL，捕获异常，与 FaultHandler 结合使用。

Sequence，定义顺序执行的流程活动。

Flow，定义并行执行的流程活动。

Invoke，调用组件服务，可调用应用的业务功能组件或别的流程。

Receive/Wait，阻塞流程，等待消息的到来，需要支持 Timeout。

Assign，变量赋值。

Reply，返回流程执行结果。

Case/While，条件分支及循环控制。

Terminate，流程结束。

Empty，空活动。

3. 系统概要设计

3.2-1 工作流引擎结构设计

在 3.1 需求分析的基础上，设计出工作流引擎结构，如图 3-8。

主要的功能组件说明如下：

Process Manager，负责对引擎中的流程实例进行管理，调度和监控。

Activity Listener，负责对流程执行过程中的事件进行监听，例如开始，结束，挂起，恢复，出错等，主要是为了对流程执行过程进行履历管理以及支持调试、模拟等功能。

Bpel Repository，负责 Bpel 流程文件的加载和解析，以及按照流程定义创建流程实例。

Activity，流程是由各种活动以及子流程组成，所以在这里 Process 也定义为了 Activity。不同的 Activity 具有不同的语法，完成不同的功能，工作流引擎根据各 Activity 定义的运行

带格式的：缩进：首行缩进： 0 字符

规则进行解释执行。

Process Instance，是流程的一个实际运行实例，它带有一个 Process Execution Context，包含流程运行所需要的运行时数据以及流程的状态。

Visitor，是为了将 Activity 的定义和执行解耦而采用了 Visitor 设计模式，BPEL 中的 Activity 的种类很多，但是相对固定，其访问执行的方法类似，都需要递归遍历其子节点，所以采用 Visitor 模式是很适合的。

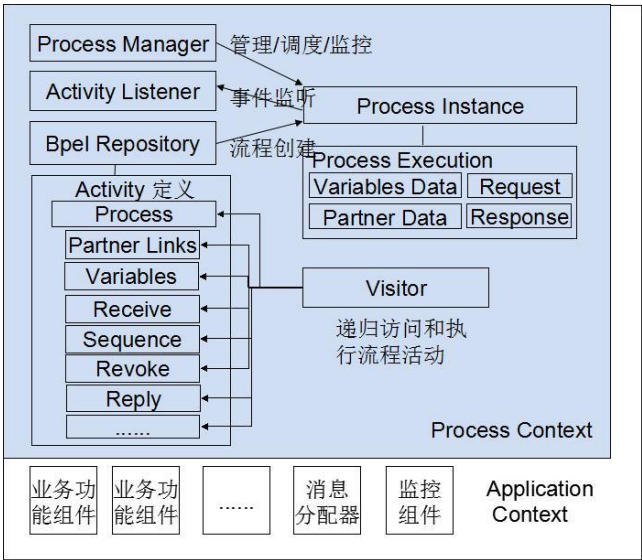


图3-8 工作流引擎架构设计

3.3-2 工作流引擎的实现和验证

3.3.2.1 工作流引擎的实现

为了将工作流引擎与应用的耦合度降低，在设计工作流引擎的架构时，对需要应用环境支持的地方都采用了接口设计，当实际需要嵌入应用系统时，需要实现这些接口。本课题采用的应用系统环境是 Java 平台的 Spring 框架，而 Spring 框架的一个优点是采用了依赖注入和控制反转的理念，所以非常容易实现松耦合。需要实现的组件有：

Process Context 接口, 将 Spring 的 Application Context 注入 Process Context 中, 这样在 Process Context 中就可以访问在 Spring IOC 容器中注册的应用业务功能组件。

Bpel Repository 接口, 因为采用了 xml 文件的方式保存 BPEL 流程文件, 所以需要利用文件系统来实现 Bpel Repository, 同时还要使用 xml 解析工具用于读取和解析流程定义文件。

消息分配器接口, 由于工作流引擎以组件的形式运行于应用系统的容器中, 消息分配器接口可以通过 Spring 的 Application Context 访问工作流引擎, 触发流程的执行。消息和流程的映射采用消息名称和流程名称匹配的方式进行。消息分配器接口与引擎的数据传输采用 xml 文档。

监听器接口, 应用可以通过实现工作流引擎提供的监听器接口来实现对工作流引擎执行事件的监控, 以满足对日志, 调试, 追踪以及模拟的需要。

3.32.2 工作流引擎的测试和验证

测试和验证的步骤大致分为三个部分:

引擎初始化: 应用容器创建工作流引擎实例, 设置流引擎的配置参数, 加载和解析 BPEL 流程文件, 将流程注册到引擎。

消息分配: 应用的接口组件接收到 ESB 的消息, 提取消息的名称, 将 ESB 的消息转换为对应的流程输入所需要的 XML 文件格式, 然后触发工作流引擎执行相应的流程。

流程创建: 工作流引擎根据流程的定义创建 Process 实例, 并用接收到的数据初始化流程的运行环境。

流程执行: 流程准备就绪后, 按照流程定义的活动进行递归执行, 需要访问业务组件时通过 Process Context 获取业务组件, 利用 Java 反射技术进行执行并返回结果。

4. 实验方案的可行性分析

在理论和技术方面: 基于 BPEL 的工作流引擎产品已经在企业中得到了广泛应用, 本方案是从适合自身企业(行业)实际情况的需要出发, 对 BPEL 规范进行了适当的剪裁, 去掉了 WEB Service 部分, 其他基本遵守 BPEL 规范, 因此理论上是完全可行的。另外, 公司在其他系统开发中也经常使用 java, xml 等技术, 大部分系统的架构都以 Spring framework 为基础, 有着丰富的应用经验, 在技术上也是完全可行的。

在经费和人力方面: 本课题的研究与应用属于公司整个应用系统升级改造中的核心部分, 得到单位的大力支持。

基于以上的分析, 该实施方案具有可行性。

批注 [雨林木风6]: 缺少学院模版中的: 4、拟采用的设计方法、
环境等; 5. 技术难度及特色分析
: 6、本人主要工作描述

四、工作进度的大致安排

应包括文献调研，工程设计，新工艺、新材料、新设备、新产品的研制和调试，实验操作，实验数据的分析处理，撰写论文等。

2014.03~2014.05 调研国内外 workflow 技术的应用和发展现状，现有 BPEL 工作流引擎产品的优缺点，深入学习和理解 BPEL 规范。

2014.06~2014.07 对 BPEL 工作流引擎进行需求分析，包括应用的上下文，主要功能，业务组件编排方式，外部接口实现方式等。

2014.08~2014.11 根据需求对 BPEL 规范进行剪裁，设计 BPEL 工作流引擎内核。

2014.12~2015.01 在 Spring 框架下实现和验证 BPEL 工作流引擎。

2015.02~2015.04 工作总结，完成研究生论文。

2015.05~2015.06 准备并完成毕业答辩。

附录 A 流程定义文件规范（DTD 文件）

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    <!DOCTYPE process SYSTEM "bpeL.dtd">
        + : 1 or more
        * : 0 or more
        ? : 0 or 1
        none : 1
-->
<!--process-->
<!ELEMENT process
    (partnerLinks | variables | faultHandlers | sequence)*
>
<!ATTLIST process
    name CDATA #REQUIRED
    targetNamespace CDATA #FIXED "http://bpeL.oasis-open.org/*/"
    xmlns:bean CDATA #FIXED "http://bean.oasis-open.org/"
    xmlns:bpeL CDATA #FIXED "http://bpeL.oasis-open.org/"
>

<!--PartnerLinks-->
<!ELEMENT partnerLinks (partnerLink)*>
<!ELEMENT partnerLink EMPTY>
<!ATTLIST partnerLink name CDATA #REQUIRED>
<!ATTLIST partnerLink partnerLinkType (bean: | bpeL:) "bean:">

<!--variables-->
<!ELEMENT variables (variable)*>
<!ELEMENT variable (#PCDATA)>
<!ATTLIST variable name CDATA #REQUIRED>
<!ATTLIST variable messageType (byte | int | long | String | org.jdom.Document) "String">

<!--expression-->
<!ELEMENT expression (function)+>
<!ATTLIST expression name CDATA #REQUIRED>
<!ELEMENT function EMPTY>
<!ATTLIST function value CDATA #REQUIRED>

<!--faultHandlers-->
```

```

<!ELEMENT faultHandlers (catch*, catchAll)>
<!-- ATTLIST faultHandlers name CDATA #REQUIRED -->

<!-- ELEMENT catch (sequence | invoke | empty)* -->
<!-- ATTLIST catch
      name CDATA #REQUIRED
      faultName CDATA #REQUIRED
-->

<!-- ELEMENT catchAll (sequence | invoke | empty)* -->
<!-- ATTLIST catchAll name CDATA #REQUIRED -->

<!-- sequence -->
<!-- ELEMENT sequence (expression | script | wait | sequence | flow | receive | invoke | reply | assign |
pick | switch | terminate | while | empty)+ -->
<!-- ATTLIST sequence name CDATA #REQUIRED -->

<!-- ELEMENT while (sequence | invoke) -->
<!-- ATTLIST while
      name CDATA #REQUIRED
      condition CDATA #REQUIRED
-->

<!-- script -->
<!-- ELEMENT script (input*, output?, snippet?, fault?) -->
<!-- ATTLIST script
      name CDATA #REQUIRED
      operation CDATA #REQUIRED
-->
<!-- ELEMENT snippet (#PCDATA) -->

<!-- empty -->
<!-- ELEMENT empty EMPTY -->
<!-- ATTLIST empty
      name CDATA #REQUIRED
-->

<!-- wait -->
<!-- ELEMENT wait EMPTY -->
<!-- ATTLIST wait

```

```

    name CDATA #REQUIRED
    timeout CDATA #REQUIRED
>

<!--terminate-->
<!ELEMENT terminate EMPTY>
<!ATTLIST terminate name CDATA #REQUIRED>

<!--empty-->
<!ELEMENT throw EMPTY>
<!ATTLIST throw
    name CDATA #REQUIRED
    faultName CDATA #IMPLIED
>

<!--switch-->
<!ELEMENT switch (case*, otherwise)>
<!ATTLIST switch name CDATA #REQUIRED>

<!ELEMENT case (sequence | flow | receive | invoke | reply | assign | pick | switch | empty | throw |
terminate)+>
<!ELEMENT otherwise (sequence | flow | receive | invoke | reply | assign | pick | switch | empty | throw
| terminate)+>
<!ATTLIST case
    name CDATA #REQUIRED
    condition CDATA #REQUIRED
>
<!ATTLIST otherwise name CDATA #REQUIRED>

<!--receive-->
<!ELEMENT receive (input*, correlation?, fault?)>
<!ELEMENT fault EMPTY>
<!ATTLIST receive
    name CDATA #REQUIRED
    timeout CDATA #IMPLIED
>
<!ATTLIST fault faultName CDATA #REQUIRED variable CDATA #IMPLIED>

<!--invoke-->
<!ELEMENT invoke (input+, output?, correlation?, fault?)>

```



```

<!--ATTLIST invoke
    name CDATA #REQUIRED
    partnerLink CDATA #REQUIRED
    operation CDATA #REQUIRED
    ignoreThrow CDATA #FIXED "false"
    fork CDATA #FIXED "false"
>
<!--ELEMENT input (#PCDATA)
<!--ELEMENT output EMPTY
<!--ELEMENT correlation EMPTY
<!--ATTLIST input variable CDATA #REQUIRED
<!--ATTLIST output variable CDATA #REQUIRED
<!--ATTLIST correlation variable CDATA #IMPLIED condition CDATA #IMPLIED>

<!--flow-->
<!--ELEMENT flow (sequence | invoke)+>
<!--ATTLIST flow name CDATA #REQUIRED>

<!--reply-->
<!--ELEMENT reply EMPTY
<!--ATTLIST reply
    name CDATA #REQUIRED
    variable CDATA #REQUIRED
>

<!--ELEMENT assign (copy)+>
<!--ATTLIST assign name CDATA #REQUIRED>
<!--ELEMENT copy (from, to)+>
<!--ELEMENT from EMPTY
<!--ELEMENT to EMPTY
<!--ATTLIST from variable CDATA #IMPLIED expression CDATA #IMPLIED>
<!--ATTLIST to variable CDATA #REQUIRED>

<!--ELEMENT pick (onMessage*, onAlarm)+>
<!--ATTLIST pick name CDATA #REQUIRED>

<!--ELEMENT onMessage (sequence | invoke | throw | terminate | empty | input)
<!--ATTLIST onMessage
    name CDATA #REQUIRED
    operation CDATA #REQUIRED

```

>

<!ELEMENT onAlarm (correlation, (throw | sequence | invoke | terminate | empty))>

<!--ATTLIST onAlarm

name CDATA #REQUIRED

timeout CDATA #REQUIRED

>

二、选题依据

1. 阐述选题背景，研究意义，或工程设计的价值和意义，该选题国内外技术应用现状和发展趋势，选题的先进性和实用性。

1.1 选题背景

随着现代农业的快速发展，食品生产加工过程中大量农业化学品的使用以及环境卫生的恶化等因素严重影响了食品安全问题，农产品安全危机频繁发生，如近年来爆发的禽流感疫情、黑心菠萝事件、早期欧美地区的疯牛病事件、致癌香蕉事件等。各类恶性食品安全事件严重影响了人们的身体健康甚至生命安全，并给各国造成了巨大的经济损失。因此，如何对农产品进行有效跟踪和追溯，已成为一个极需迫切解决的全球性课题。在完善法律，加强监督之外，实现从农田到餐桌的全过程追溯，最为有力的举措就是建立农产品的可追溯供应链体系^[1]，从而提供可靠真实的农产品信息，满足消费者的知情权和选择权，解决信息不对称问题。与农产品追溯体系比较完善的国家相比，当前我国在整个农产品的生产过程中应用追溯系统的实例寥寥无几，全程可跟踪供应链尚未形成。因而，建立完善的农产品可追溯供应链系统具有切实的现实意义与需求。

RFID（射频识别：radio frequency identification）是一种非接触式的自动识别技术，它通过射频信号自动识别目标对象并获取相关数据^[2]。因其高效、快速、可靠、非视距读取等优点，该项技术已被广泛应用于物流、供应链、车辆识别、门禁系统、图书管理、资产管理等领域，将 RFID 技术应用于农产品追溯供应链系统可充分发挥和利用该技术的优点。基于 RFID 的农产品追溯供应链系统以 RFID 技术为载体，实现整体业务流程包括农产品从种植生产、采摘加工、运输仓储到最后的销售整个流程中的信息可视化管理，并通过记录生产、加工、质检、运输、仓储和销售等环节的数据，从而生成每个农产品包装上的唯一可追溯标签，消费者可通过该标签条码进行产品溯源查询，实现农产品的安全可追溯性。

1.2 工程设计价值意义

Den Ouden 和 ZuurBier 等学者于 1996 年首次提出了食品供应链（Food Supply Chain）概念^[3]。直至今日，食品供应链已得到极大程度的发展与完善，不少发达国家已经建立了较为成熟的农产品跟踪追溯系统。随着我国加入 WTO 后，将会有更多农产品出口到欧美地区，

然而为了符合欧美对食品跟踪与追溯的要求，更好地为出口服务，促进我国农产品质量的提高，就迫切需要实施农产品的跟踪与追溯体系。

传统的农产品环境监测跟踪追溯多采用的是人工实地检测获取环境数据，这种方式存在很大的局限性和误差性。人工检测即耗费人力、物力又受主观因素的影响，测量数据难免有误差，并且记录形式单一，不利于长久存储。新兴的物联网技术很好的解决了这些问题，通过无线传感器网络可实时采集田间信息，如空气温湿度、光照强度、土壤湿度与盐碱度等环境信息，满足了农产品跟踪追溯系统的自动化与智能化。

在跟踪追溯供应链系统中，信息是最为关键和重要的环节。传统的方法是通过手工记录完成，其直接后果就是效率低下、出错率高并且不能长久保存。RFID 作为一种新兴无接触的无线传输技术，它适用于各个领域的跟踪追溯系统。RFID 采用非接触式批量读取信息，该项特性降低了标签本身的磨损，加长使用寿命，同时更主要的是加快了流通速度，提高供应链的效率。基于 RFID 的农产品供应链系统的建立，将会促进 RFID 在供应链管理中的全面应用，一方面有利于 RFID 技术的发展和普及性的应用，另一方面也有利于发挥供应链的作用，提高供应链管理的效率，提高供应链整体及各企业成员的竞争力，使得 RFID 供应链系统在未来的追溯机制中更具有潜力和优势。

1.3 国内外技术应用现状

1.3.1 供应链管理研究现状

在供应链概念研究上，美国学者 Elise Golan 认为可追溯系统是指在整个加工过程中整个供应链体系中对某种产品特性进行追踪记录的体系，同时根据可追溯体系的自身特性设定了衡量可追溯体系的三个标准——深度、宽度和精度^[4]。我国学者方炎等将“食品可追溯”定义为食品供应链各个阶段信息流的连续性保障体系，同时将食品安全追溯制度的内容分为五个部分，包括记录管理、查询管理、目标管理、标识管理、责任管理和信用管理^[5]。

在供应链系统的建立上，国外学者 Salaun 和 Flore 认为，可追溯体系的建立有助于食品相关企业加强对食品供应链的管理，增强食品质量控制力度，从而提高相关企业的品牌信任度和树立良好口碑，可追溯制度将为供应链合作企业带来一系列的收益^[6]。日本在食品供应链管理方面处于国际领先地位，不但制定了食品安全法规，而且建立了“食品身份证制度”

产品履历和跟踪监视制度，要求生产、流通等各部门采用 RFID 电子标签技术、条码技术用来详细记载产品流通过程和生产环节中的数据信息^[7]。国内很多学者也提出了要进行食品可追溯体系的建设，其中有王波等研究了农产品和食品领域可追溯系统的技术现状，着重给出了条码自动识别技术和无线射频识别技术的应用现状和发展^[8]；谢菊芳等运用二维条码技术、无线射频技术和组件技术，设计了基于.NET 的猪肉安全生产追溯系统^[9]；张兵等将网络信息化技术与蔬果种植业的实际相互结合，使用 EAN/UCC 条码作为蔬菜产品的标识码，构建出以中心管理系统为主，生产基地管理子系统、配送中心管理子系统、公众查询子系统为辅的蔬果质量可追溯系统，进而实现蔬果业“从农田到餐桌”的全程质量控制^[10]。

然而，供应链管理作为一种全新的管理模式，涉及到一个或者多个企业的方方面面，是一个非常复杂的管理体系。从我国当前的供应链管理现状来看，这种崭新的管理模式还存在许多问题，如整体利益与企业利益的矛盾关系；信息数据库建设的进一步完善；供应链管理技术的进一步完善实施；整体性控制平台的缺乏等。

1.3.2 RFID 技术研究现状

RFID 射频识别技术在 20 世纪 60 年代开始得到发展，到 90 年代这项技术开始进入商业应用阶段。经过多年发展，RFID 技术标准化问题日趋为人们所重视，RFID 产品也逐渐得到广泛应用并且种类更加丰富多样，成本也在不断降低。目前，业界最关注的是位于中高频段的 RFID 技术，特别是 860MHz-960MHz（UHF 超高频段）的远距离 RFID 技术发展最快^[11]。

RFID 技术最先在美国得到快速发展，美国政府也是 RFID 应用的积极推动者，在 RFID 标准的建立、相关硬件技术的开发与应用领域均走在世界前列。从全球产业格局来看，目前 RFID 产业主要集中在 RFID 技术应用比较成熟的欧美市场。飞利浦、西门子、ST、TI 等半导体厂商基本垄断了 RFID 芯片市场；IBM、HP、微软、SAP、Sybase、Sun 等国际巨头抢占了 RFID 中间件、系统集成研究的有利位置；Alien、Intermec、Symbol、Transcore、Matrics 等公司提供 RFID 标签、天线、读写器等产品设备。相对于欧美等发达国家和地区，我国在 RFID 产业上的发展还较为落后。我国 RFID 缺乏关键核心技术，特别是在超高频 RFID 方面，国内发展较晚，技术相对欠缺，从事超高频 RFID 产品生产的企业很少，更缺少具有自主知识产权的创新性企业。同时，在 RFID 产业链上，包括芯片设计、标签封装、读写设备的设

计和制造、系统集成、中间件、应用软件等，我国还未形成成熟的 RFID 产业链，产品的核心技术基本掌握在国外公司手里，但是目前国内已有极少数企业已经具备了超高频读写器设计制造能力。

因为 RFID 的非接触快速读取的特点，目前已在各个领域得到广泛应用，欧美地区 RFID 技术已广泛应用到包括交通、车辆管理、身份识别、生产线自动化控制、仓储管理及物资跟踪等领域；零售商沃尔玛、制造商吉列、强生、宝洁等知名企业也积极响应 RFID 应用。国内 RFID 技术也在包括物流、零售、制造业、服装业、医疗、防伪、资产管理、动物识别、图书馆等众多领域开始广泛应用，对改善人们的生活质量、提高企业经济效益、加强公共安全以及提高社会信息化水平具有重要影响。

1.3.3 基于 RFID 供应链系统研究现状

国外对追溯供应链系统研究较早，多数学者从技术层面对追溯供应链系统的模型架构和开发进行研究，主要应用 RFID 信息技术，采用 EPCIS 编码方案，应用领域包括牛奶、水产品、果蔬、生猪及豆制品和粮油食品等。其中，Mira Trebar 等提出养殖鱼类供应链 RFID 追溯系统^[12]；Gianni Fenu 等提出基于 RFID 技术的供应链可追溯系统，并应用于生猪及其制品中^[13]；Filippo Gandino 等提出粮油食品行业通过集成 RFID 应用于可追溯管理^[14]。美国成立家畜标识小组（USAIP），共同参与制定家畜标识工作计划，采用 RFID 电子标签实现对畜产品生产过程的追溯；日本在蔬果 EDI 协议会的主持下，开展了苹果、米、蔬菜等农产品基于 RFID 技术的生产履历和食品跟踪系统的建立。

国内学者研究食品供应链追溯系统所应用的信息技术主要包括条形码技术、RFID 技术、RFID 技术与编码方案结合、物联网关键技术、3G 技术、虹膜识，采用的编码方案有 EAN/UCC 和 EPCIS，应用的领域包括水产品、果蔬、生猪、大宗农产品等。其中，罗敏等提出将 RFID 技术、无线数据通信技术、网络技术及数据库技术等应用于食品安全检测追溯系统的设计^[15]。张土前、蒋平安等提出以新疆阿克苏苹果为例，将 GIS 技术、数据库技术、NET 技术和 JSP 等技术相结合，开发了基于 RFID 与 WebGIS 的阿克苏苹果质量安全溯源系统^[16]。

目前，关于追溯系统的研究侧重于生猪及肉制品、家禽、水产品、大宗农产品等，现

有的食品供应链追溯系统的研究主要是基于 RFID 技术的模型架构和系统设计的构建，真正系统实现的很少。鉴于此，本系统主要是在国内外相关研究的基础上，在确定食品供应链中各环节溯源信息的基础上，实现以 RFID 电子标签为数据载体对食品进行唯一标识，实现数据的自动采集，增强个体标识的精度和准确性，设计基于 RFID 物联网架构下农产品供应链追溯系统，实现农产品供应链全程信息可视化管理和信息可追溯。

1.4 发展趋势

1.4.1 供应链管理发展趋势

随着社会经济的发展和市场竞争的趋势日益激烈化，不断发展和完善供应链管理已成为企业提高自身市场竞争力的新型手段，物联网背景下的供应链管理也在实践中出现了一些新的发展趋势^[17]：

1) 敏捷化供应链：敏捷供应链强调从整个供应链的角度考虑、决策和效绩评价，追求快速反应市场需求，提高供应链各环节效益，实现利益共享的双赢目标。同时尽量迅速、准确的满足顾客的个性化、多样化的需求，不断的培养并提高顾客忠诚度，从而拥有较为稳定的顾客资源。

2) 绿色化供应链：绿色供应链引入全新的设计思想，对产品从原材料购买、生产、消费，直到废物回收再利用的整个供应链进行生态设计，通过链中各个企业内部部门和各企业之间的紧密合作，使整条供应链在环境管理方面协调统一，达到系统环境最优化。

3) 集成化供应链：借助物联网，企业能够有效满足供应链管理中各个环节的信息共享，更好地让供应链环节中各个部门之间协调化发展，实现供应链的集成化管理。

4) 智能化供应链：运用物联网的高效信息识别和定位跟踪优势，企业可以有效实现对生产环节中的信息快速、准确的获取，对运输配送环节中的物流、车辆实时跟踪追溯，强化企业生产、运输等环节的智能化管理，从而提供更加高品质的产品。

5) 虚拟企业化供应链：虚拟企业化供应链管理是将整个供应链看作是一个虚拟的企业，供应链中的所有企业看作是企业中的一个部门。从每个成员企业追求自身利益最大化调整到追求整个供应链管理的最优化来实现链条上每个成员成本最小化和利益最大化。供应链上的各相关企业通过充分发挥各自的核心能力，从而形成优势互补，更好为客户提供服务。

1.4.2 RFID 技术发展趋势

在未来的几年中，RFID 技术将继续保持高速发展的势头。电子标签、读写器、系统集成软件、公共服务体系、标准化等方面都将取得新的进展。随着关键技术的不断进步，RFID 产品的种类将越来越丰富，应用和衍生的增值服务也将越来越广泛。

RFID 芯片设计与制造的发展趋势：芯片功耗更低、作用距离更远、读写速度更快、可靠性更高、成本更低等，芯片技术与应用系统整体解决方案紧密结合。

RFID 读写器设计与制造发展趋势：读写器发展具有多功能、多接口、多制式，并向模块化、小型化、便携式、嵌入式方向发展。同时，多读写器协调与组网技术也是未来的发展趋势。

RFID 系统集成软件将向嵌入式、智能化、可重组方向发展，同时 RFID 技术与条码、生物识别等自动识别技术，以及与互联网、通信、传感网等信息技术融合，海量 RFID 信息处理、传输和安全对 RFID 系统的集成和应用提出新的挑战。

1.5 选题先进性和实用性

1.5.1 先进性

相对于条形码标签信息，RFID 采用非接触式，信息读取的时候不需要接触到标签本身，降低了身份标签的磨损。同时，RFID 可以批量读取食品产品的信息，比条形码在单位时间内可以读取更多的产品信息^[18]。RFID 自动采集技术可从农蔬的种植、采摘、质量检测、运输配送、销售各个环节进行数据自动化采集，为用户提供全程实时动态跟踪查询。

基于 RFID 的供应链管理系统在食品供应链中提供完全透明的管理能力，保障食品安全全程可视化控制、监控与追溯，同时为各级管理者提供真实、有效、及时的管理和决策信息，为业务的快速发展提供支撑，摆脱传统的人工管理模式，并可以全面监控种植养殖源头污染、生产加工过程的添加剂以及有害物质、流通环节中的安全隐患。为有可能出现食品安全隐患的环节进行有效评估和科学预警提供依据，让企业管理者做科学决策提供了坚实的数据保障，从而全面降低成本，提高利润和企业竞争力。

基于 RFID 的农蔬供应链管理系统利用 RFID 先进技术并依托网络技术、数据库技术、通信技术，实现信息融合、查询、监控，数据通过网络实现实时、准确传输，通过 Web 方式

消费者即可方便查询所购买食品的详细完整的追溯信息。

1.5.2 实用性

基于 RFID 的农蔬供应链管理系统以 RFID 标签作为信息载体在整个农产品供应链上满足信息的快速、准确、实时传输与追溯，对供应链上的各个环节都充分发挥了其优势。

生产环节上，RFID 技术实时记录农产品的详细种植信息，如种植品种、播种记录、灌溉记录、施肥记录、病虫害防治记录以及肥料信息等，建立农产品的生殖档案并保存到生产管理系统的数据库中。加工环节上，对农产品进行分批包装，并在包装上贴上 RFID 标签，该标签与农产品的生殖档案进行关联。仓储环节上，通过安装在仓库的龙门架 RFID 阅读器，可快速读取批量的农产品信息，建立入库信息；当出库时，同样可以快速读取批量 RFID 信息，提高了效率也减少了出错率。运输环节上，通过对货物的扫描建立与运输单以及司机车辆的信息关联，从而达到责任可追溯的目的。销售环节上，通过智能信息采集设备，对销售的农产品建立销售记录，同时消费者可通过追溯码在公共服务平台上查询产品从生产、包装、运输、仓储及销售等详细信息。

2. 主要参考文献（列出作者、论文名称、期刊名称、出版年月）。

[1]王喜成,郑海峰.基于 RFID 的农产品追溯系统应用研究.TP 391.44

[2]互动百科: RFID.<http://www.baike.com/wiki/RFID>

[3]Den Ouden,PJP Zuurbier.Vertical co-operation in agricultural production-marketing chains-with special reference to product differentiation in pork[J]. Agribusiness,1996,12 (3): 277-290.

[4]Elise Golan,Barry Krissoff,Linda Calvin and Kenneth Nelson.Traceability in the USA food supply:Economic Theory and Industry Studies,Agricultural Economic report No.830.3/2004.

[5]方炎,高观,范新鲁,陈华宁.我国食品安全追溯制度研究[J].生态经济,2007,(4).

[6]Salaun,Flore.The regulation of food safety and the use of traceability/tracing in the EU and USA:convergence or divergence Speeches at food safety conference Washington.DC,March 2001.

[7]黎继子,周德翼,刘春玲等.论国外食品供应链管理和食品质量安全[J].外国经济与管理,2004(12).

- [8]王波,王顺喜.农产品和食品领域可追溯系统的研究现状[J].中国安全科学学报,2007,17.
- [9]谢菊芳,陆昌华.基于 NET 架构的安全猪肉全程可追溯系统实现[J].农业工程学报,2006,22.
- [10]张兵,黄昭瑜,叶春玲等.蔬菜质量安全可追溯系统的设计与实现[J].食品科学,2007,8.
- [11]闫娜.低功耗低成本无源射频识别标签芯片的研究与设计[D].博士学位论文.
- [12]Mira Trebar,Andrej Grah,Alejandro Alvarez Melcon etal.(2011)Towards RFID traceability systems of farmed fish supply chain,1-6.In SoftCOM 2011,19th International Conference on Software,Telecommunications and Computer Networks.
- [13]Gianni Fenu, Paolo Garau: RFID-based supply chain traceability system. ©2009 IEEE
- [14]Filippo Gandino, Bartolomeo Montrucchio, Maurizio Rebaudengo, Erwing Ricardo Sanchez: On Improving Automation by Integrating RFID in the Traceability Management of the Agri-Food Sector. IEEE Transactions on Industrial Electronics 56(7): 2357-2365 (2009).
- [15]罗敏,韩慧敏,何瑞赣,黄利勇,张嗣忠.基于 RFID 的食品安全监测追溯系统的设计[J]2012.
- [16]张土前,蒋平安,邹鹏.基于 RFID 与 WebGIS 的阿克苏苹果质量安全溯源系统[J].新疆农业科学.2012(08).
- [17]桑娟萍.关于物联网的供应链管理发展趋势研究[J].中国商贸,2012(1).
- [18]蒙少东,许福才.食品供应链可追溯制度的国际比较及其启示.经济问题,2007(2).

三、课题内容及具体方案

1. 课题内容

基于 RFID 的农蔬供应链管理系统致力于解决农产品从生产环节开始，经过包装环节、运输环节、仓储环节一直到最终的销售环节的整个供应链信息可视化管理和信息可追溯。采用先进的 RFID 技术、传感器网络技术、数据库技术以及通信传输技术等开发一整套集合信息采集、信息存储、信息传输与信息查询与一体的智能供应链管理系统。

本课题包含以下方面的内容：

- 1) 系统架构的搭建：整个系统分为三层结构，数据采集子系统、应用服务子系统、公共服务信息平台系统；其中应用服务子系统包含四个独立的子模块：生产子系统、运输子系统、仓储子系统和销售子系统。
- 2) RFID 数据采集：本系统采用三种设备进行 RFID 信息的采集，分别是手持机、RFID 读写器、电子秤设备，每种设备采集的数据都要经过应用程序处理后传输到各个子系统中。
- 3) 应用子系统模块开发：应用子系统包含有四个子模块，分别是生产、运输、仓储和销售子模块，每个模块都采用 Web 服务的形式进行开发。
- 4) 公共服务追溯平台搭建：追溯平台的搭建主要是提供给消费者进行产品信息的追溯查询，属于公共服务平台，同样采用 Web 的形式进行开发。
- 5) 通信协议与数据传输格式：系统中的数据通信主要包含两个方面，一个是底层采集系统将采集的 RFID 信息传输到应用子系统中，一个是应用子系统将产品信息传输到公共追溯平台上；其中第一个采用基于 Socket 的 UDP 协议进行数据传输，第二个采用 Web Service 服务的方式进行数据的传输。

2. 系统需求分析（应用软件工程专业描述工具描述）

2.1 概述

基于 RFID 供应链系统致力于解决农产品从种植—采摘—加工—运输—仓储—销售的整个流程中的信息可追溯，从而满足最终消费者的可追溯查询产品的整个生命线信息。同时满足产品信息的可视化管理，提供 Web 服务接口供企业管理者使用与操作。

2.2 功能需求

2.2.1 公共服务信息平台

公共服务信息平台在本系统中又可叫做产品追溯信息平台，以 Web 网页形式进行信息展示，主要功能是供消费者追溯查询具有 RFID 或条形码标识的产品信息，消费者可输入产品的 RFID 或条形码号进行相关的产品信息查询。

2.2.2 应用服务子系统

应用服务子系统处于公共服务平台系统和数据采集系统的中间层，该层通过自行制定的协议标准接收从数据采集层通过读写设备采集到的产品信息数据，并负责对数据进行解析和保存在本地数据库中。该层是基于 SSH 框架开发的 Web 管理系统，提供对产品信息进行存储、添加、查询、修改、删除等操作的功能。同时该层也服务于公共服务平台系统，当消费者追溯查询时，公共服务平台系统从该层的各个子模块获取产品信息进行展示。该层包含四个独立的子模块：生产子模块、运输子模块、仓储子模块和销售子模块。

2.2.2.1 生产子模块

生产子模块中主要是对农产品的整个生产过程进行管理，主要包括种植信息管理、采摘信息管理、包装信息管理以及单品信息管理。其中，种植信息和采摘信息由管理员根据记录信息手动输入到系统中，包装信息及单品信息由采集子系统通过 RFID 读写器及手持机扫描器采集到的条码信息并通过网络协议根据自行定义的格式传递到生产子系统中，生产子系统同时将产品的种植、采摘信息跟包装和单品信息进行绑定，满足生产信息可追溯。

生产子模块系统采用 B/S 架构开发，以 Web 网页形式提供操作界面，采用最常用的 SSH 框架技术和 Weblogic 容器作为应用服务器。系统主要包含有四个用户权限，包括系统管理员、种植管理员、采摘管理员和包装管理员，不同的用户有相应的管理权限和操作权限。生产子系统启动时监听 socket 端口，并随时接收采集子系统传递的采集数据，并同时接收到的数据进行解析和存储；同时生产子系统提供一个 Web Service 调用接口，供服务平台系统调用以获取生产子系统的产品生产信息。生产子系统用例图如图 2.1 所示：

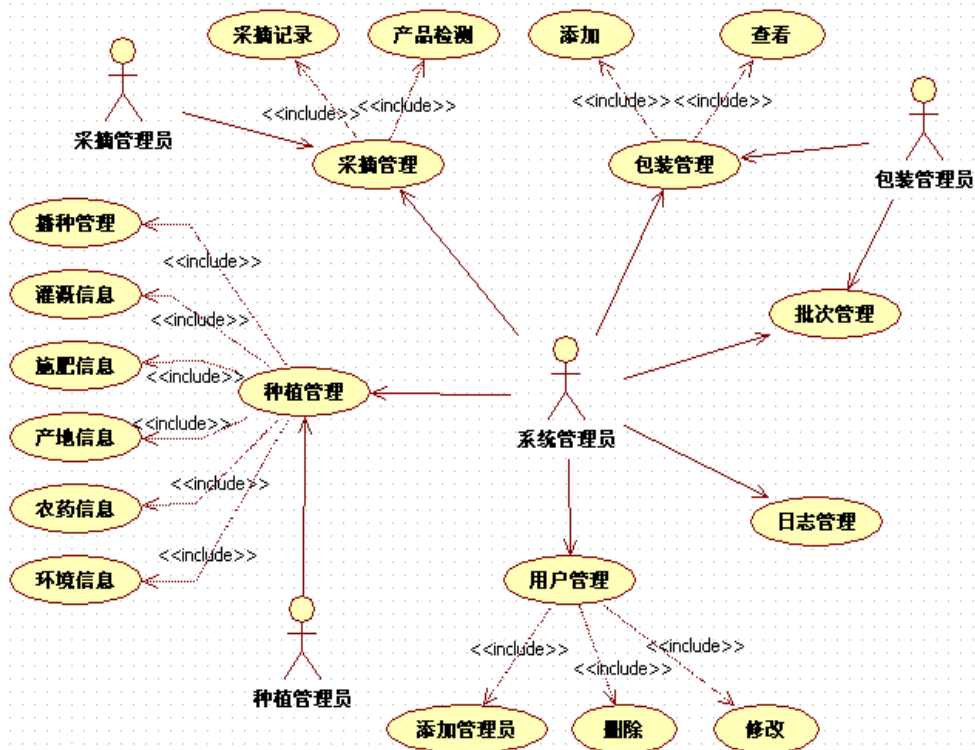


图 2.1 生产子系统用例图

2.2.2.2 运输子模块

运输子模块主要对货品在运输过程中的信息管理，包括车辆信息管理、车次信息管理、运输单管理、运输货物信息管理等。

运输子模块系统采用 B/S 架构开发，以 Web 网页形式提供操作界面，采用最常用的 SSH 框架技术和 Weblogic 容器作为应用服务器。运输子系统启动时监听 socket 端口，同时运输子系统提供一个 Web Service 调用接口，供服务平台系统调用以获取运输子模块系统的产品运输信息。运输子模块系统用例图如图 2.2 所示：

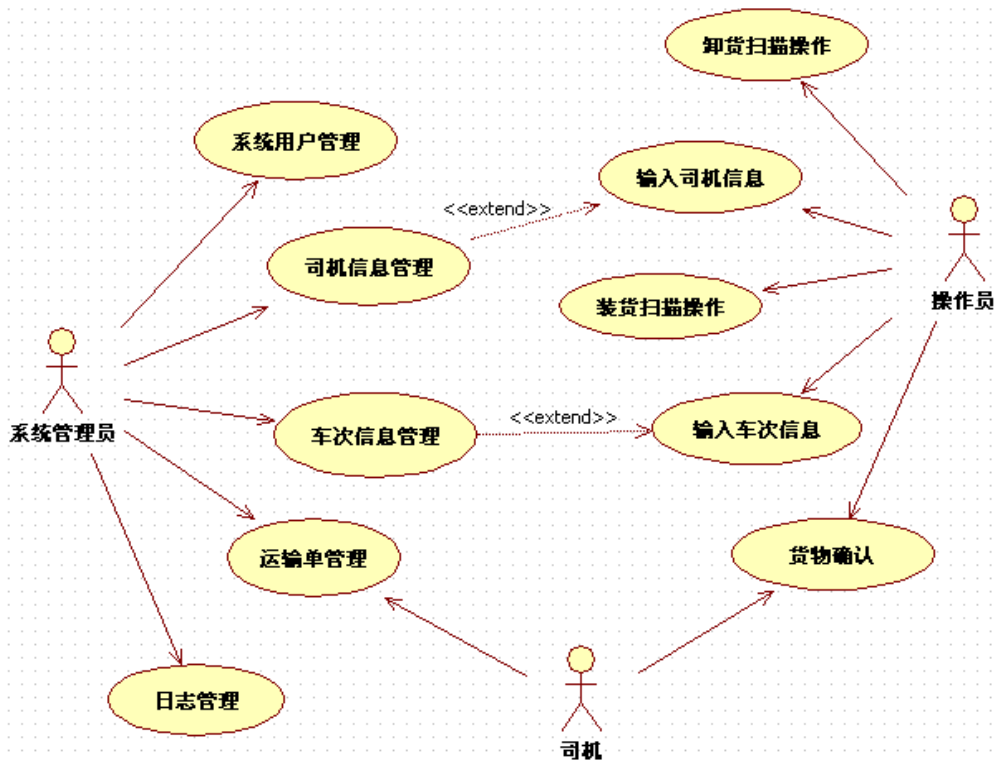


图 2.2 运输子模块系统用例图

2.2.2.3 仓储子模块

仓储子模块主要对货品的存储信息管理，包括仓库信息管理、入库信息管理、出库信息管理、气调信息管理、货品盘点等功能。

仓储子模块系统采用 B/S 架构开发，以 Web 网页形式提供操作界面，采用最常用的 SSH 框架技术和 Weblogic 容器作为应用服务器。仓储子系统启动时监听 socket 端口，同时仓储子系统提供一个 Web Service 调用接口，供服务平台系统调用以获取仓储子模块系统的产品仓储信息。仓储子模块系统用例图如图 2.3 所示：

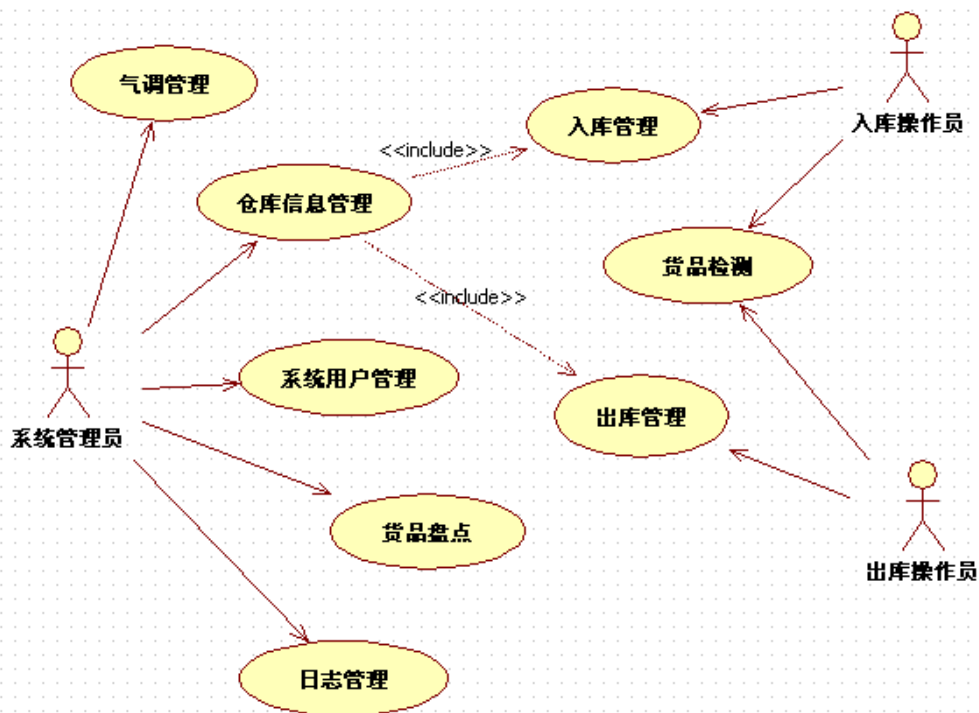


图 2.3 仓储子模块系统用例图

2.2.2.4 销售子模块

销售子模块中主要对产品最终的销售流程及销售信息进行管理，包括售货员信息管理、会员信息管理、销售单信息管理、销售数据统计分析等功能。销售过程中使用电子秤作为信息采集工具，电子秤上安置 RFID 读写器设备，当销售货品放置到电子秤上时自动扫描货品上的 RFID 标签，称量货品重量，并把数据传递到 Wince 应用系统中，Wince 应用系统同时将数据进行汇总整理并转换成一定格式通过 UDP 通信协议传递到销售子系统中，销售子系统接收数据并解析存储到本地数据库中。

销售子模块系统采用 B/S 架构开发，以 Web 网页形式提供操作界面，采用最常用的 SSH 框架技术和 Weblogic 容器作为应用服务器。销售子系统启动时监听 socket 端口，同时销售子系统提供一个 Web Service 调用接口，供服务平台系统调用以获取销售子系统的产品销售信息。

销售子系统用例图如图 2.4 所示：

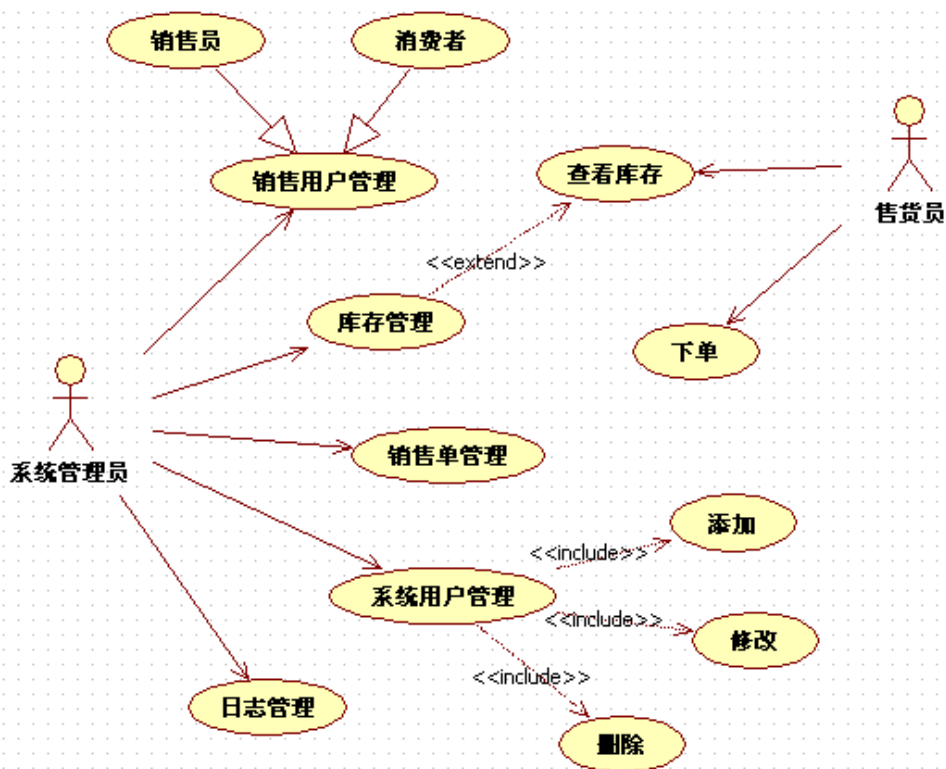


图 2.4 销售子系统用例图

2.2.3 数据采集子系统

2.2.3.1 生产数据采集

生产数据采集主要是产品在包装过程中进行装箱操作后贴上 RFID 标签，并通过读写器设备进行扫描，将 RFID 标签信息或者条码信息传递到 Wince 应用程序中，Wince 应用程序将批量 RFID 信息与产品包装信息打包成一定格式再传递到生产子模块中。生产数据采集时序图如图 2.5 所示：

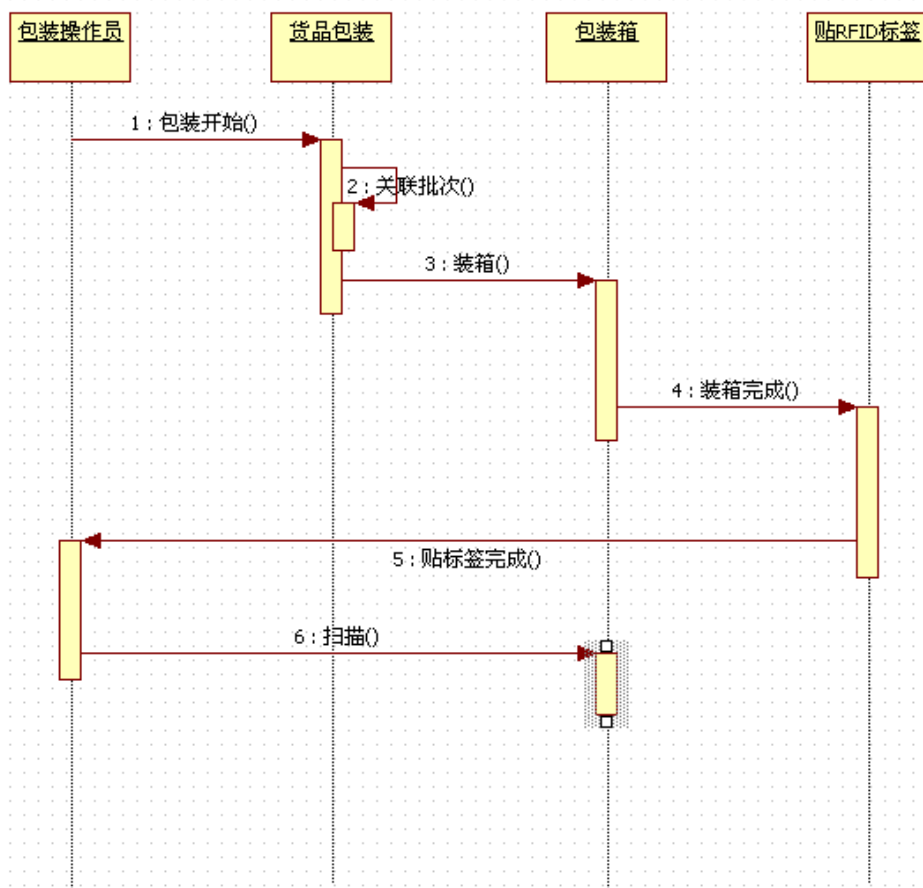


图 2.5 生产数据采集时序图

2.2.3.2 运输数据采集

运输数据采集主要是在货品装车时通过 RFID 读写设备扫描待运输货品的 RFID 信息，记录司机、目的地、车辆等信息并通过 Wince 应用程序传递到运输子模块中；当货品到达目的地卸货时，再次通过读写设备进行扫描，并将扫描到的 RFID 信息与装货时进行对比，确认是否有丢失或损坏现象，并将信息传递到运输子模块中。装车数据采集时序图如图 2.6 所示：卸车数据采集与装车数据采集类似。

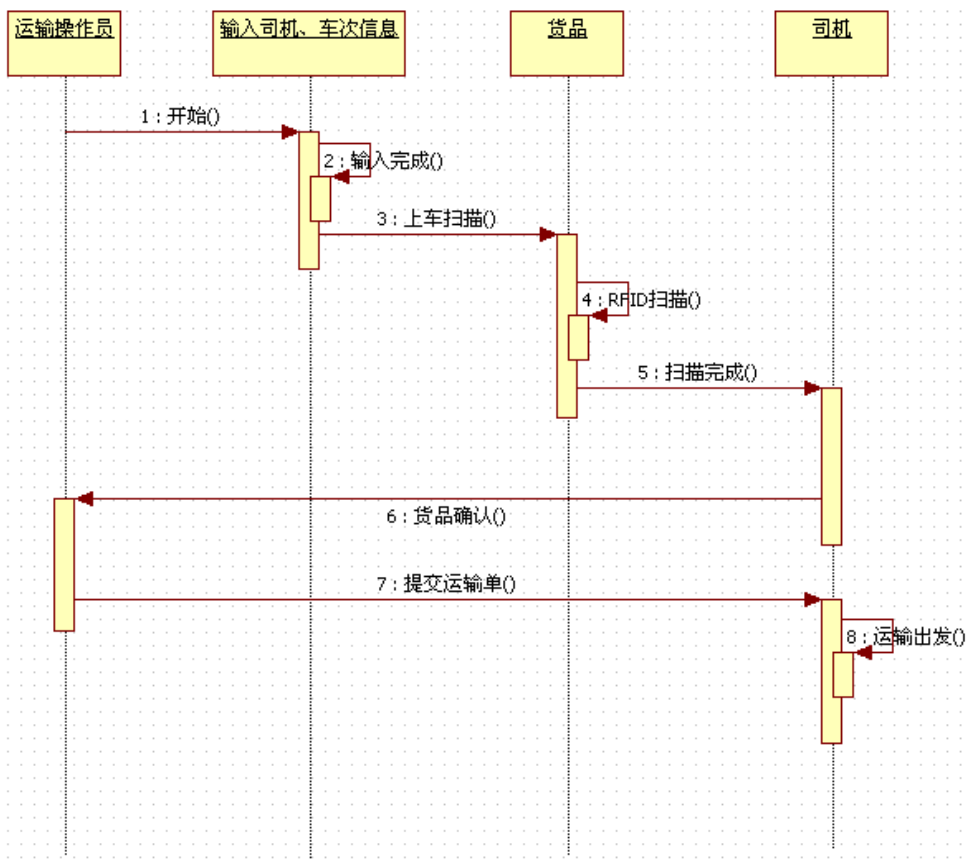


图 2.6 装车数据采集时序图

2.2.3.3 仓储数据采集

仓储数据采集包括入库、出库信息采集，通过仓库龙门架自动读取货品 RFID 信息，同时记录入库、出库时间等信息，通过 Wince 应用程序将货品出库、入库信息传递到仓储子模块中。入库数据采集时序图如图 2.7 所示：出库数据采集与入库数据采集类似。

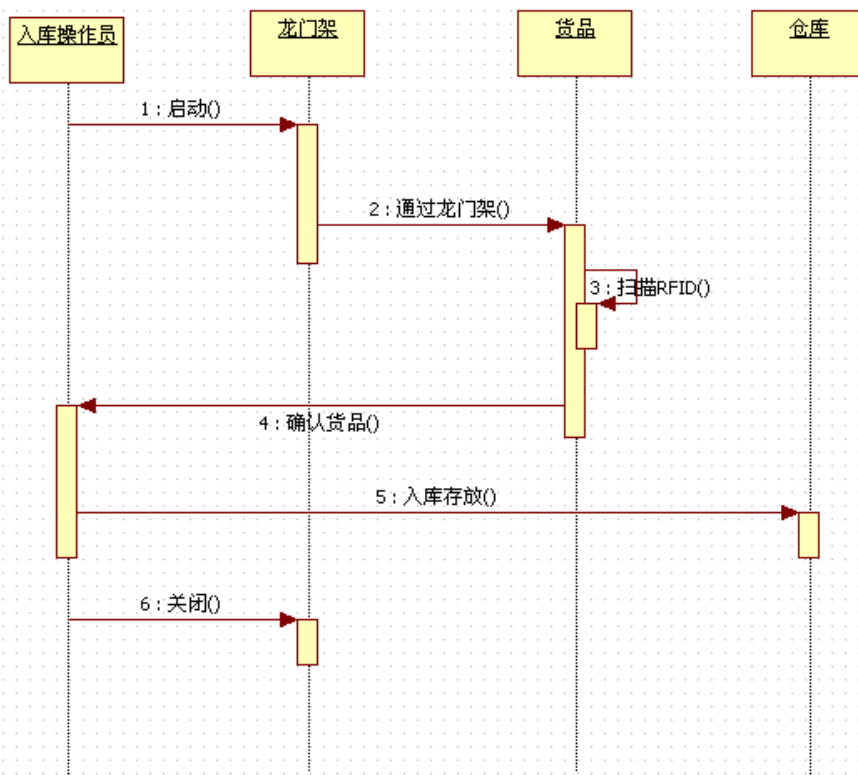


图 2.7 入库数据采集时序图

2.2.3.4 销售数据采集

1) 电子秤：通过在电子秤上安置 RFID 读写设备扫描货品 RFID 信息，并称量货品重量，通过电子秤接口将信息传递到 Wince 应用程序中，Wince 应用程序将销售信息传递到销售子模块中。

2) 手持机：售货员通过手持机扫描货品 RFID 信息，通过手持机接口传递到 Wince 应用程序中，在 Wince 应用程序界面上手动输入货品价格、数量等信息并传递到销售子模块中。

销售数据采集时序图如图 2.8 所示：

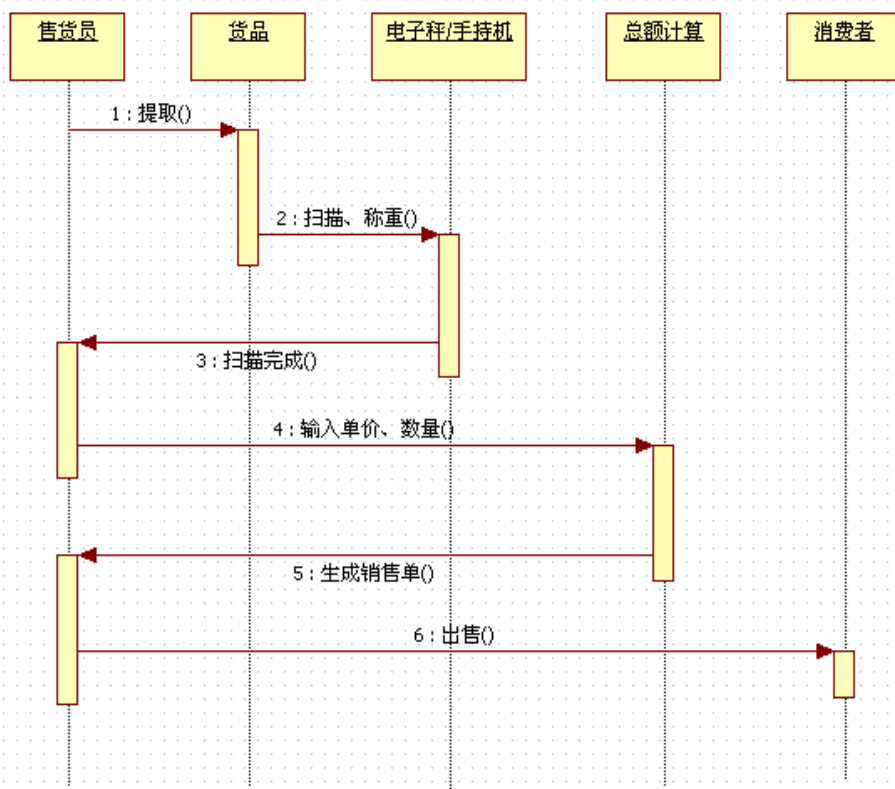


图 2.8 销售数据采集时序图

2.3 性能需求

2.3.1 农产品批量 RFID 信息采集装置

设计一个装置，对农产品包装箱的 RFID 信息进行批量式采集，提高物流过程中的装车、卸车以及仓储过程中的出库、入库信息采集效率。技术指标如表 2-1 所示：

表 2-1 采集装置性能指标

| 项目 | 指标类型 |
|-------------|----------|
| 硬件设施 | 龙门架 |
| CPU | ATOM/ARM |
| RFID Reader | EPC GEN2 |
| 通讯方式 | LAN |

| | |
|----|---------|
| 显示 | 悬挂式 LCD |
|----|---------|

2.3.2 农产品销售交易智能感知装置：设计一个具有称量计价、产品信息自动采集、交易控制的智能终端，满足农产品在批发、零售市场销售交易的快速信息采集需求。技术指标如表 2-2 所示：

表 2-2 销售智能装置性能指标

| 项目 | 指标类型 |
|-------------|-------------------|
| CPU | ARM9 |
| RFID Reader | ISO14443/EPC GEN2 |
| Bar Code | Code128 |
| 控制方式 | 面板 |
| 组网方式 | LAN |
| OS | WINCE/Linux |
| 数据接口 | USB |
| 秤体量级 | 500Kg/50Kg |
| 电子秤标准 | GB/T 75xx 标准 |
| 显示 | LCD |
| CPU | ARM9 |

2.3.3 农产品供应链信息系统：基于 Web 架构的农产品生产、运输、仓储、销售信息管理系统，通过各种环节的 RFID 信息采集设备将数据传输到子系统中，构建可追溯的供应链信息。同时满足消费者在公共服务平台的查询服务。技术指标如表 2-3 所示：

表 2-3 供应链信息系统指标

| 项目 | 指标类型 |
|------|-------------------|
| 数据中心 | Oracle 数据库 |
| 软件架构 | SSH、Weblogic 容器 |
| 运行模式 | 7*24 运行模式、系统延时<1s |
| 性能要求 | 系统延时<1s、系统并发数>512 |

2.4 外部接口需求

2.4.1 硬件接口

1)手持设备接口：负责读取条形码信息并传输到应用服务子系统中。

2)Reader 接口：负责读取 RFID 信息并根据协议规定的格式传输到应用服务子系统中。

3)电子秤接口：负责读取 RFID 信息、测量货品重量、读取高频卡信息并传输到应用服务子系统中的销售子系统。

4)无线传感器网络接口：负责读取环境信息并传输到应用服务子系统中。

2.4.2 通信接口

1)采集系统与应用子系统通信接口：采用基于 Socket 的 UDP 传输协议，根据自行定义的数据格式进行传输。

2)应用子系统与公共服务平台通信接口：采用 Web Service 服务的方式，由应用子系统提供调用接口，公共服务平台调用接口方法获取 Xml 数据。

2.5 质量需求

2.5.1 界面交互需求

系统 Web 界面要求风格统一、界面元素一致，交互过程流畅，用户理解无障碍；手持机界面要求简洁清晰，操作简单；龙门架设备要求操作简单易懂，能快速准确处理批量 RFID 信息。

2.5.2 系统健壮性及稳定性需求

健壮性是指软件对于规范要求以外的输入情况的处理能力。农产品的物流管理设计很多复杂的业务流程，其中涉及对系统稳定性要求较高的数据大批量传输、实时查询等需求，特别是在批量扫描过程中，可能会有漏掉、重复扫描等情况的发生，这就要求系统具有复查流程；对于传输数据不一致、格式错误或不符合要求等情况，系统需要能够较好的处理并给出具体的错误提示信息。因此，整个系统需要具有较好的健壮性，对于运行过程中产生的错误和异常要有足够的处理能力。

3. 系统概要设计（应用软件工程专业描述工具描述）

3.1 系统概述

农蔬产品的可追溯依赖于产品从生产、包装、储运到销售整个过程的信息记录，要实现一

个追溯平台需要建立一个跟踪体系，采集和记录农蔬产品各个环节的信息。

基于 RFID 农蔬供应链系统是以 RFID 作为农蔬产品与信息系统联系的桥梁，将农蔬产品的生产、加工、包装、储运、销售与信息系统有效整合在一起，提高效率的同时也方便了产品追溯。系统整体应用采用三层架构实现，公共服务平台系统、应用子系统和数据采集子系统，各层之间通过网络协议和 Web 服务进行相互通信，最终达到可追溯的目的。

3.2 软件结构设计

3.2.1 系统整体应用架构

系统的整体架构图如图 3.1 所示：

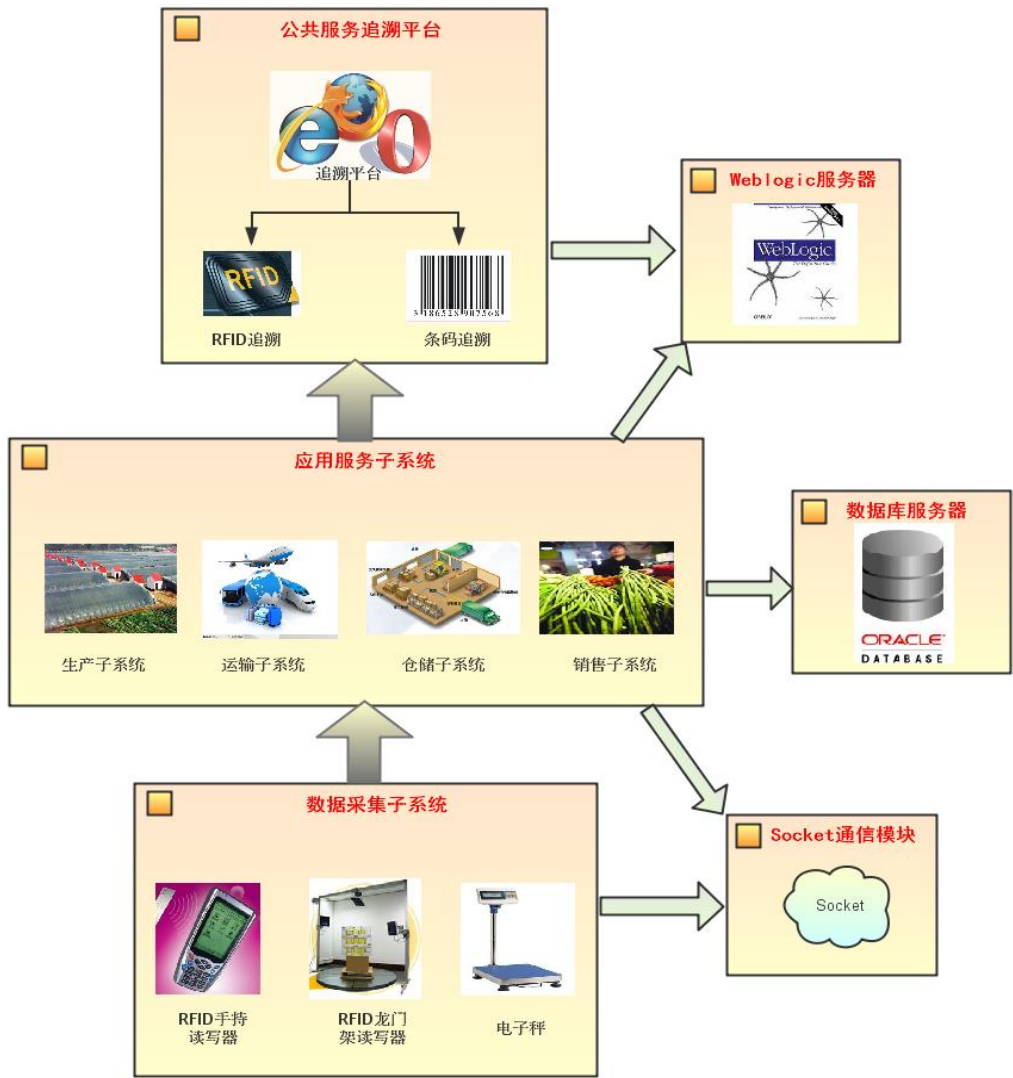


图 3.1 系统架构图

系统部署节点包括数据库服务器、用户计算机、Weblogic 服务器、手持机 RFID 读写器、龙门架读写器、电子秤设备，各节点描述如表 3-1 所示。

表 3-1 系统部署节点

| 名称 | 需求数量 | 描述 |
|--------------|------|--|
| 数据库服务器 | ≥1 台 | 数据库服务器为系统提供各类数据的持久化 |
| 用户计算机 | 多台 | 通过浏览器访问 Weblogic 服务器，对应用服务子系统的四个模块进行管理操作，同时可进入公共服务平台进行追溯查询 |
| Weblogic 服务器 | 1 台 | Weblogic 服务器上部署所有的 Web 服务，对 Web 应用服务进行统一管理 |
| 手持机读写器 | ≥1 台 | 生产环节中手持机读写器通过扫描包装 RFID 信息，将产品的生产信息与 RFID 进行绑定 |
| 龙门架读写器 | ≥2 台 | 在运输及仓储环节，龙门架读写器扫描批量的 RFID 信息，并将产品的运输及仓储信息与 RFID 进行关联 |
| 电子秤设备 | ≥1 台 | 在销售环节，电子秤扫描货品 RFID 信息，将销售信息与 RFID 进行关联 |

3.2.2 公共服务平台端

公共服务平台采用 B/S 架构，提供两个查询接口，同时调用 Web Service 的 API 接口执行子系统的查询方法获取相应信息进行展示，系统架构图如图 3.2 所示：

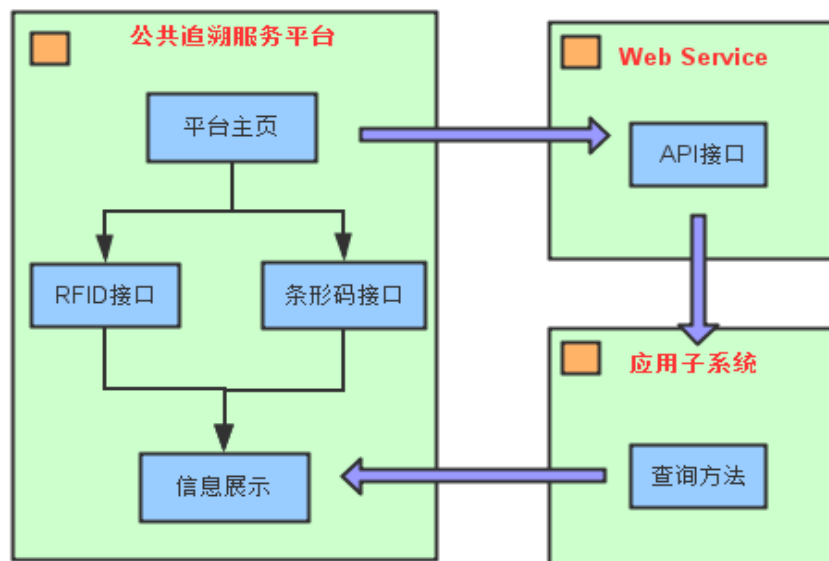


图 3.2 追溯平台架构图

3.2.3 应用子系统端

应用子系统采用 B/S 架构开发，使用 SSH 框架技术，软件架构图如图 3.3 所示：

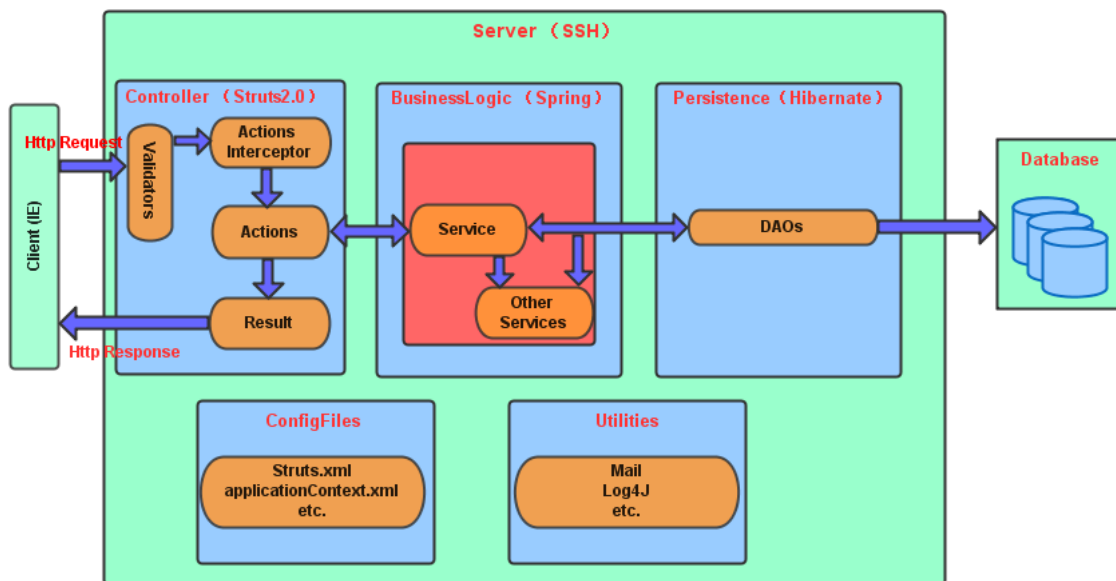


图 3.3 子系统软件架构图

SSH 框架从职责上可分为三层结构：表示层、业务逻辑层、数据持久层。在表示层中，首先通过 JSP 页面实现交互界面，负责传送请求（Request）和接受响应（Response），然后 Struts

根据配置文件（struts.xml）将 ActionServlet 接收到的 Request 委派给相应的 Action 处理；在业务逻辑层中，管理服务组件的 Spring IoC 容器负责向 Action 提供业务模型（Model）组件和该组件的协作对象数据处理（DAO）组件完成业务逻辑，并提供事务处理、缓冲池等容器组件以提升系统性能和保证数据的完整性；在持久层中，依赖于 Hibernate 的对象化映射和数据库交互，处理 DAO 组件请求的数据，并返回处理结果。

3.2.4 数据采集端

数据采集系统主要是通过硬件设备采集产品的 RFID 信息，其中可能包含多种硬件采集设备，这就需要使用 RFID 中间件技术来处理不同采集设备的数据传输和数据格式问题。其系统架构图如图 3.4 所示：

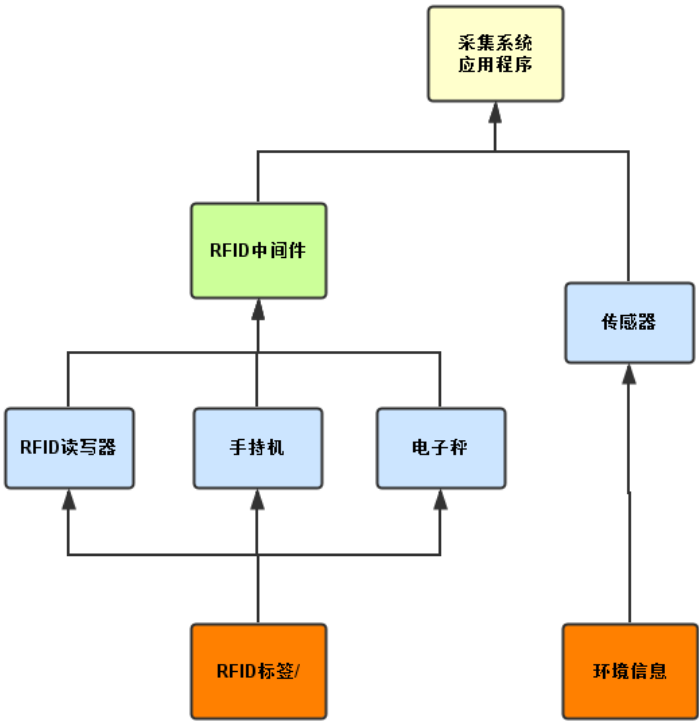


图 3.4 采集系统架构图

3.3 系统结构设计

基于 RFID 的农蔬产品供应链可追溯系统致力于解决产品信息的可追溯功能，本系统中的产品信息管理主要是在应用层的四个子模块中体现，本模块也主要对应用子系统四个子模块给出进行详细的结构设计描述。

四个子模块都是采用 Web 开发模式,使用 SSH 框架技术,每个项目的包组成结构按照 SSH 框架技术进行设计,项目包图如图 3.5 所示:

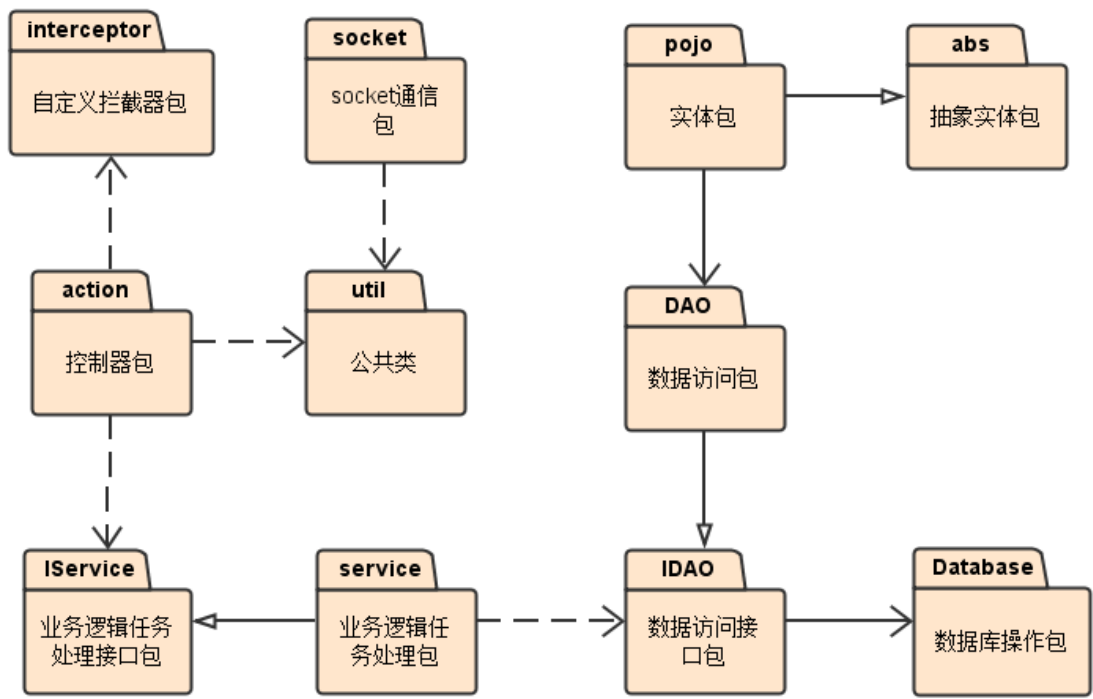


图 3.5 系统包图

其中,实体包 pojo 中存放每个实体的实体类,该实体类中定义各个实体的构造方法以及扩展方法,而实体的属性定义在实体的抽象包 abs 中;数据访问接口包 IDAO 中存放着对每个实体的业务逻辑方法类,每个方法类在 DAO 包中的对应类中进行实现;数据库操作包 Database 中定义了连接、关闭数据库方法的类;业务逻辑任务接口包 IService 中定义了功能业务方法类,它为 Action 提供调用接口,具体实现则在 service 包中对应的实现类中,业务逻辑处理依赖于 IDAO 包中数据访问方法类;控制器包 Action 中为每个用户行为定义方法类,Action 接收用户请求并调用 Service 方法完成相应的业务处理,最终将请求结果返回给用户;公共类 util 包中定义了公共调用方法类;socket 通信包中定义了 socket 通信方法类。

3.3.1 生产子系统

生产子系统中,不同的角色用户拥有不同的管理权限,通过定义一个用户 (User) 类和一个角色 (Role) 类,一个用户实例分配一个角色。其中,种植用户管理种植信息,种植信息包括播种信息、环境信息、产地信息、施肥信息、用药信息和灌溉信息;采摘用户管理采摘信息,

同时包含有产品检测信息；包装用户管理包装信息，同时包括单品信息；管理员拥有所有权限，同时还包括日志信息。生产子系统的类图如图 3.6 所示：

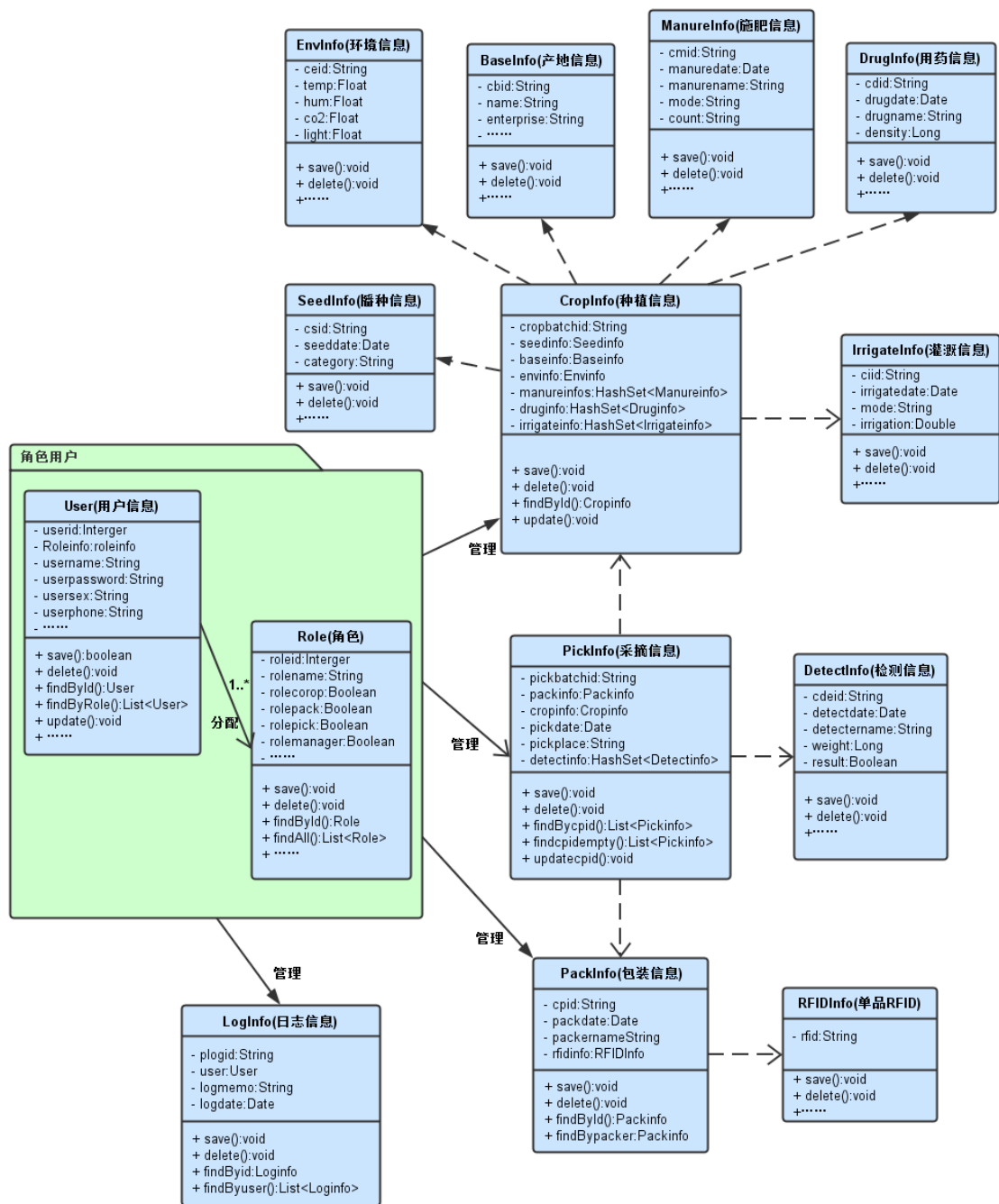


图 3.6 生产子系统类图

3.3.2 运输子系统

运输子系统中，同样根据不同的管理权限分配给用户不同的角色。其中，运输操作员管理

运输信息，包括运输货物信息等；司机用户管理车次信息和运输信息，车次信息包括车辆信息和运输环境信息；系统管理员拥有所有权限，还包括有日志信息管理。运输子系统的类图如图 3.7 所示：

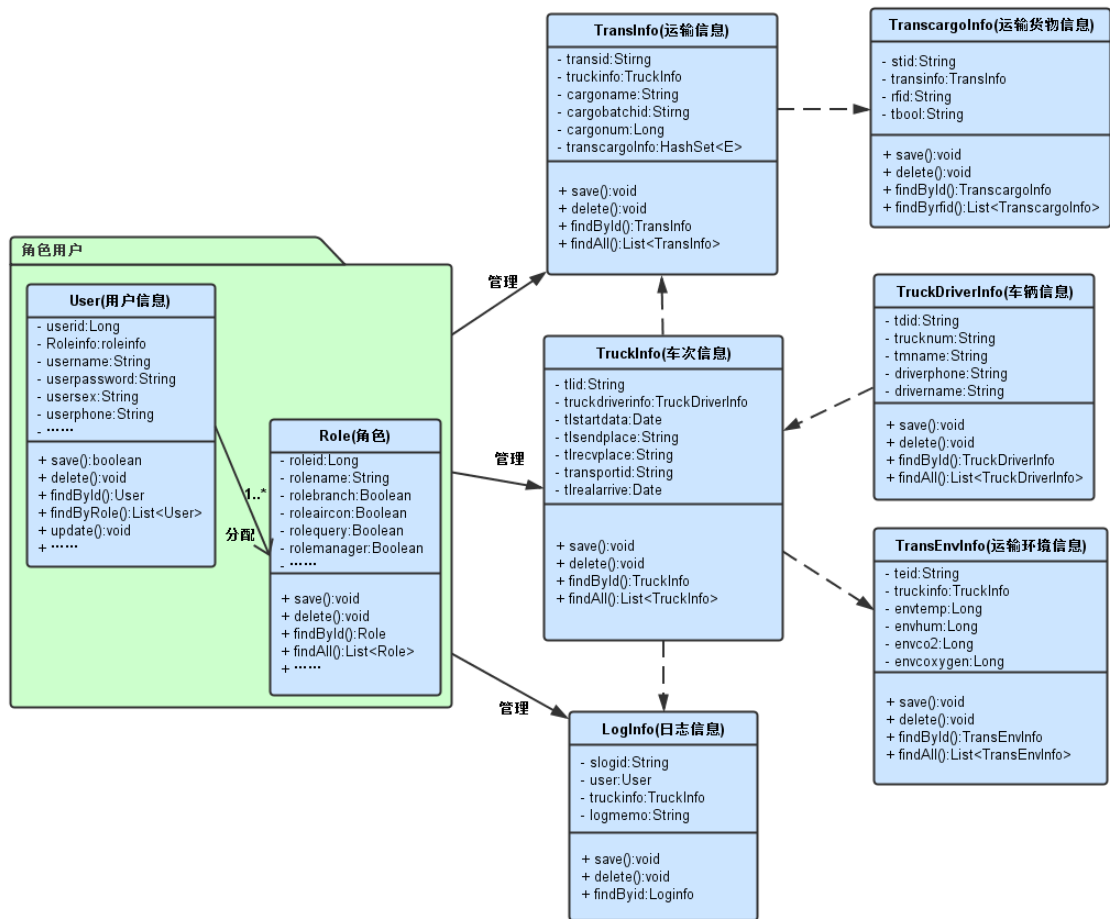


图 3.7 运输子系统类图

3.3.3 仓储子系统

仓储子系统中，同样是根据不同的系统管理权限分配给用户不同的角色。其中，入库管理员主要管理货物的入库信息；出库管理员主要管理货物的出库信息；仓库管理员管理仓库信息，包括入库出库信息以及仓库的环境信息等；系统管理员拥有所有的权限，还包括日志信息管理。仓储子系统的类图如图 3.8 所示：

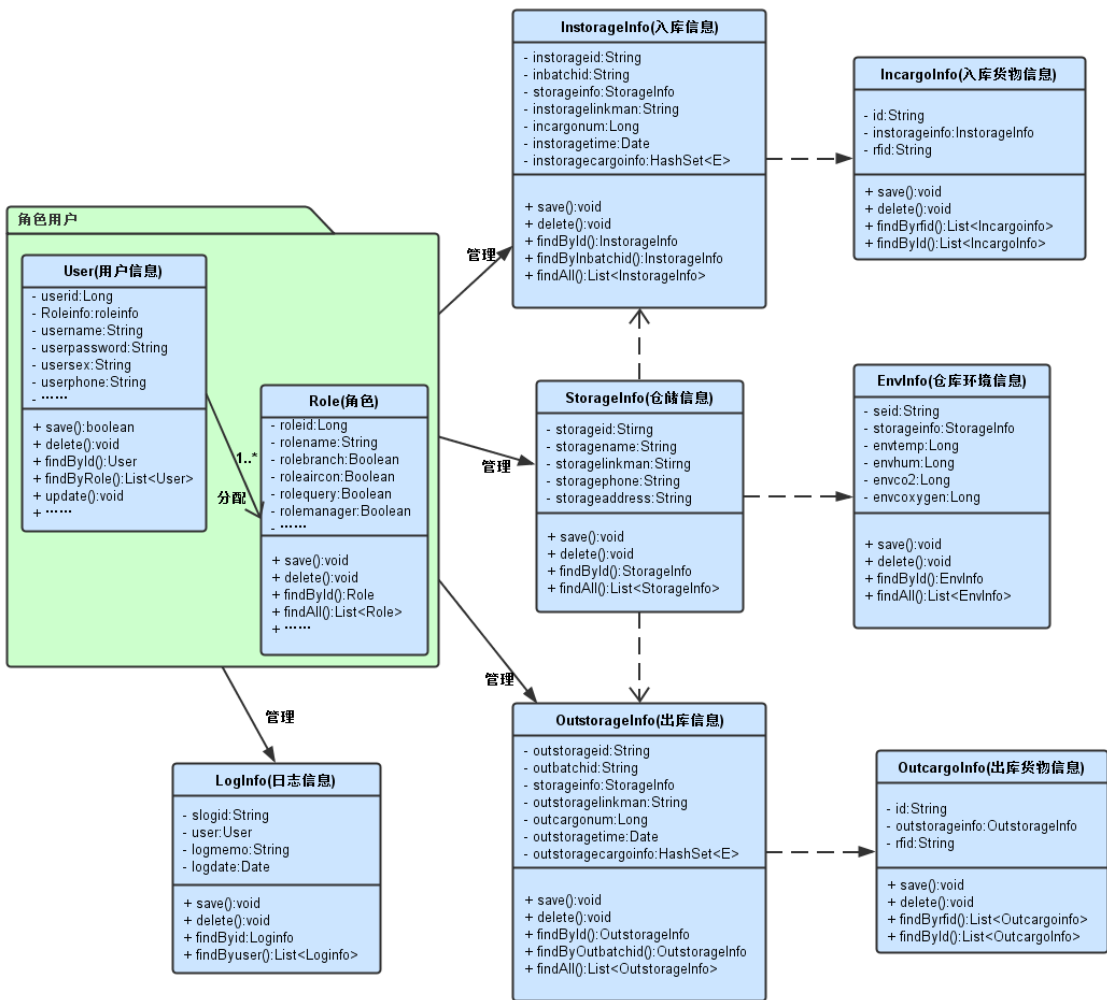


图 3.8 仓储子系统类图

3.3.4 销售子系统

销售子系统中，同样也是根据不同的权限给用户分配不同的角色。其中，销售员用户管理销售记录信息，包括单品信息；销售管理员用户管理销售单信息，包括销售记录信息、买家信息以及卖家信息等；系统管理员拥有所有权限，还包括日志信息管理权限，销售子系统的类图如图 3.9 所示：

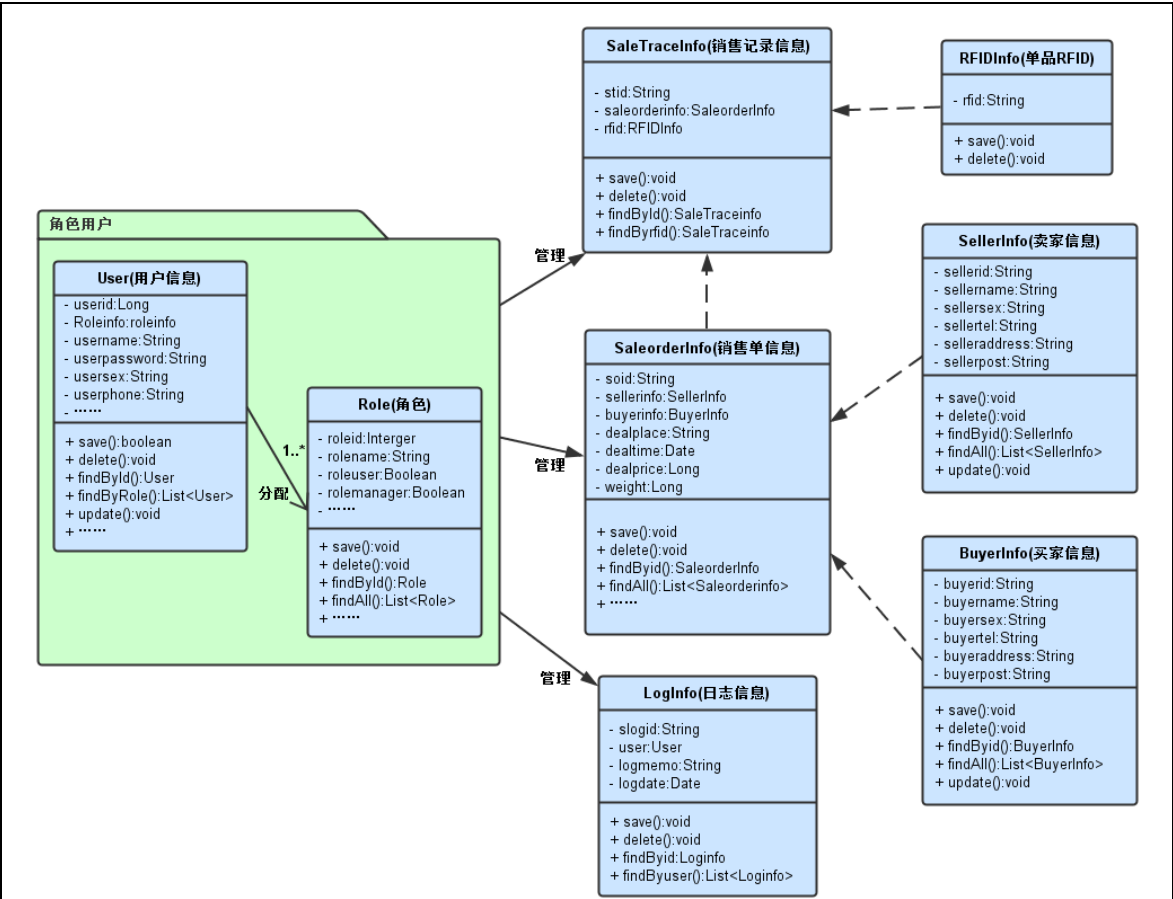


图 3.9 销售子系统类图

3.4 通信协议设计

3.4.1 采集数据通信设计

采集数据通信是采集子系统通过硬件设备采集到 RFID 信息后上传到上层的四个子系统中，通信协议采用的是 UDP 传输协议，具体格式如下所述：

1、包格式：

| | |
|-----------|-----|
| 包头（10 字节） | 数据包 |
|-----------|-----|

2、包头格式：

| | |
|---------|---|
| 类型 | 格式 |
| 客户端请求 | seek+packagekind(2 字节) |
| 服务器应答请求 | ack+packagekind(2 字节) |
| 客户端发送数据 | packagekind(2 字节)+packagenum(2 字节)+currentnum(2 字 |

| | |
|----------|--|
| | 节)+length(2 字节) |
| 服务器应答数据包 | ack+packagekind(2 字节)+currentnum(2 字节) |

其中 packagekind:表示包类型（01-生产、02-运输、03-仓储、04-销售）、packagenum:表示包总数、currentnum:表示当前包序号、length:表示包长度。

3、数据包格式：

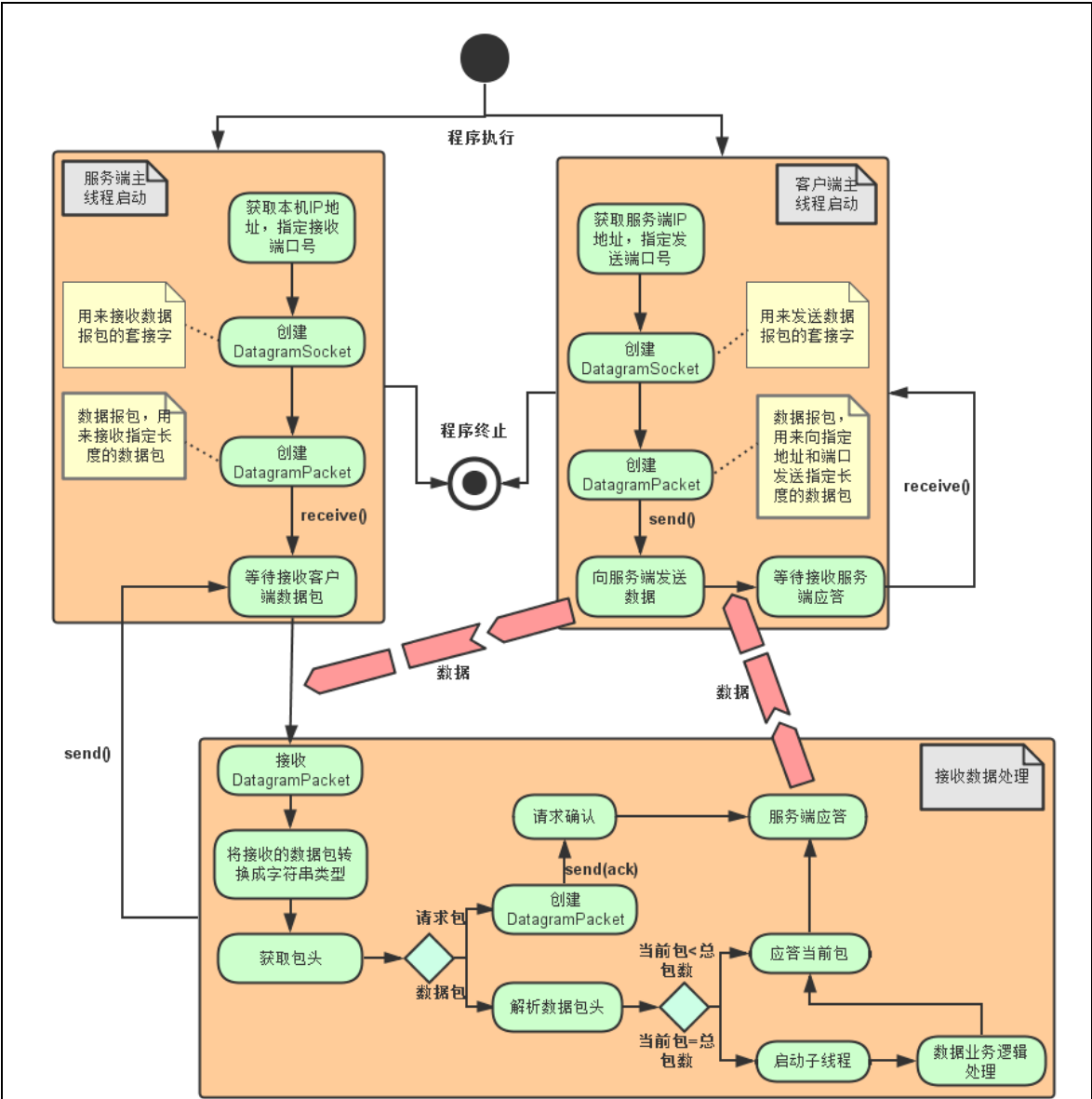
| 类型 | 格式 |
|--------|---------------------------|
| 客户端数据包 | 包类型+包总数+当前包+包长度+Json 数据格式 |
| 服务端数据包 | 接收到的当前包 |

4、通信过程描述：

服务端由于是部署在 Weblogic 服务器上，当 Weblogic 服务启动时会激活所有部署在其上的应用程序，本系统启动的同时服务端的 socket 主线程也会启动。首先服务端的通信模块的主线程会获取本机的 IP 地址并指定一个事先定好的接收端口，程序利用 DatagramSocket 封装类来发送和接收数据报包的套接字，利用 DatagramPacket 封装类来接收数据包，创建好套接字后调用其 receive()方法接收客户端数据，此方法会阻塞程序执行并一直监听接收端口，当客户端有数据发送过来，服务端接收到数据后启动子线程解析数据，同时发送确认数据包并继续监听接收端口。

客户端即采集模块程序，程序获取硬件设备采集到的数据后，将数据进行整理并转换成一定格式，当数据量大时，进行分包处理。客户端首先向服务端发送一个请求包（seek+包类型），当收到服务端确认（ack+包类型）后，将数据包加上包头进行发送，包头共 8 个字节，分别是：包类型（2 字节）+包总数（2 字节）+当前包序号（2 字节）+包长度（2 字节），客户端同时接收服务端发送的确认包（ack+包类型+当前包序号）。

整个 socket 通信状态图如图 3.10 所示：



3.4.2 追溯数据通信设计

追溯数据通信是用于应用服务的四个子系统向公共服务平台传递产品信息数据，因为两个平台都是采用 Web 形式，所以使用 Web Service 传输数据会更加方便快捷。传输过程主要是先将需要上传的数据信息进行序列化，并转换成 Xml 格式通过网络传输到公共追溯平台系统，由平台系统进行数据的解析处理并最终展示在页面上供消费者查询。

4. 拟采用的开发方法、环境，测试方案等

4.1 开发方法

系统开发拟采用面向对象的分析及设计方法，开发平台采用易移植的 Java 平台，便于后期的维护与移植，系统框架采用较成熟的 SSH 框架技术，通信协议采用 UDP 协议，通信方式采用 socket 套接字和 Web Service API 接口。

4.2 开发环境

开发环境及使用工具如表 4-1 所示：

表 4-1 开发环境及工具

| 项目 | 开发工具 |
|---------|--|
| 操作系统 | Microsoft Windows XP、Microsoft Windows Server 2003 |
| 设计工具 | StarUML、Power Designer、Microsoft Viso 2010 |
| 开发工具 | MyEclipse 10、Microsoft Visual Studio 2010 |
| Web 服务器 | Weblogic10g |
| 数据库管理系统 | Oracle11g |
| 硬件工具 | 手持机、RFID 读写器、龙门架、电子秤 |

4.3 测试方案

系统测试方案及工具如表 4-2 所示。

表 4-2 测试方案及工具

| 测试项目 | 测试方法 | 测试工具 |
|------|------|------------|
| 单元测试 | 白盒测试 | JUnit4 |
| 集成测试 | 白盒测试 | JUnit4+Ant |
| 性能测试 | 黑盒测试 | LoadRunner |
| 前段测试 | 白盒测试 | QUnit |

5. 技术难度及特色分析

5.1 技术难度

1) 追溯编码与追溯信息的映射关系

由于农产品在生产过程中尚未产生追溯码，只是记录日常的农事管理信息，在农产品经过采摘、包装之后才会产生追溯码。因此，在数据库设计时需要考虑如何建立农产品的生产操作

信息（种植、灌溉、施肥、用药、检测等）、生产环境信息（温度、湿度、光照、土壤等）、农产品追溯码的映射关系。

2) 数据交互

如何把硬件采集到信息一步一步传输到最终的追溯平台上供消费者查看，这中间有两级的数据传输，一个是采集子系统把采集的 RFID 信息传输到应用子系统中，同时将产品的信息与 RFID 信息进行关联保存到各自的数据库中；另一个是应用子系统将数据传输到追溯平台上供消费者查询。考虑到其数据传输的特点，分别采用 socket 通信方式实现采集子系统与应用子系统的的天数据传输和采用 Web Service 技术实现追溯平台系统与应用子系统之间的数据交互。其中 Web Service 能够提供一个通过 Web 进行调用的 API，它为应用程序之间的互操作提供了分布式的应用平台。应用子系统中的采集信息上传过程中，主要是先将信息序列化，并以 Xml 格式上传到追溯平台进行解析处理并展示。

3) 产品追溯

如何为消费者提供快速、可靠的产品信息是本系统的一个技术难点也是追求的目标，由于平台系统本身并不会直接存储产品的所有信息数据，当消费者在执行查询操作时，平台系统需要立即向应用子系统发出 Web Service 请求，应用子系统执行查询操作并上传数据。在整个过程中如何提高查询速度以及数据传输速度是一个需要解决的问题，本系统采用 Web Service 服务的方式，通过 Xml 格式进行数据传输，很大程度上提高了信息传输效率，但还有待进一步的改进完善。

5.2 特色分析

1) 现有的供应链系统中，不少方法在技术层面上只采用单一的 RFID 和条码技术，没有形成一个具有 RFID 和智能传感器联合的物联网架构的供应链系统。本系统在生产、运输及仓储环节采用 RFID 和传感器网络联合的物联网架构，更加具有实时性和快速反应能力。

2) 整个系统采用三层架构设计，每层都作为单独的模块进行开发，只提供每层之间的通信接口，这样开发效率大大提高，并且每层之间相互独立存在，具有很强的扩展性和便于后期维护。

3) 供应链管理中，由于要对整个流程进行信息处理，往往上下游之间关系不确定，特别是

运输和仓储环节，流程具有很强的不确定性；并且上下游之间存在着信息交互过程，当上游信息断层会直接导致整个追溯链的断链；本系统中，对每个环节看做独立的子模块，以 RFID 作为整个供应链的连线，每个模块都根据 RFID 为主键建立信息档案，这样很好解决了上述的问题，并且系统模块化更容易扩展和维护。

6、本人主要工作描述

- 1) 公共服务平台系统设计与开发
- 2) 应用子系统中的生产子模块和销售子模块开发设计
- 3) 通信模块设计与开发
- 4) 数据采集子系统集成测试
- 5) Web Service 调用接口开发设计
- 6) 系统集成与部署

四、工作进度的大致安排

应包括文献调研，工程设计，项目开发和调试，实验数据的分析处理，撰写论文等。

2013.11 – 2013.12 对供应链系统及实现方法进行调研国内外相关技术及发展状况，初步设计系统实施方案；

2013.12 ~ 2014.01 确定需求分析，完成系统结构设计、数据库设计，搭建系统实施开发环境；

2014.01 ~ 2014.04 开发公共追溯平台系统和应用子系统的四个模块，并进行单元测试、集成测试并修正系统错误，完成开题报告编写；

2014.04 ~ 2014.07 完成整个系统的集成，并部署在 Weblogic 服务器上，进行集成测试

2014.07 ~ 2014.09 优化代码结构，对代码进行详细注释与检错；同时对各项功能进行测试与完善，收集材料编写文档，同时撰写毕业论文，准备毕业答辩。

五、预期成果

应包括软硬件产品、文档、模型、专利、论文等

1. 软件产品：

- 公共服务追溯平台系统
- 四个应用子系统

2. 硬件产品：

- 手持机设备
- 龙门架读写器
- 电子秤设备

3. 文档论文：

- 系统设计文档、操作文档
- 硕士毕业论文 1 篇

二、选题依据

| |
|--|
| <div>1. 阐述选题背景，研究意义，或工程设计的价值和意义，该选题国内外技术应用现状和发展趋势，选题的先进性和实用性。</div> <div>1.1 选题的研究意义</div> <div><p>我国是一个农业大国，也是一个人口大国，解决众多人口的吃饭问题，是一个非常重要的问题，然而，我国传统农业发展模式存在诸多不利的因素。首先，我国的农业资源匮乏。我国的农业资源总量在全球各国中居于前列，然而从人均占有量而言，却比世界平均水平低，且近几年有呈现逐年减少的趋势。其次，频发的极端恶劣天气对农业产生一定的负面影响。近几年，中国境内特别是部分农作物的主产区遭遇了极端恶劣天气。由于农业自身的弱质性，造成其抗击外界冲击的能力非常弱，不仅是自然灾害，病虫害也会对农业产生严重影响。最后，农业整体的利润水平较低。长期以来，农业生产环节利润水平不高都是不争的事实，一方面是由于延续了多年的以农业支持工业发展的国家发展策略造成的长期影响，另一方面也是由于绝大多数农业生产者分散而力量薄弱，且进入门槛较低，并不具备产业链的议价能力。此外，原材料、农资产品价格普涨，化肥、棚膜、柴油的价格在过去的几年里都有比较明显的价格上涨，推高了农业生产的成本。</p><p>要解决上述问题，需要引进先进技术，集约化地使用农业资源，转变农业经营管理方式，采用信息化技术，提高农业现代化水平，提升农业劳动生产率。本文就是基于这一应用需求，通过利用传感器技术、无线通信技术、嵌入式技术、计算机技术等，构建一个智能大棚系统，实现对农作物生长过程的全面实施监控，让大棚可以完全按照所种植作物的生长规律和要求，调整环境参数，及时检测作物生长的土壤状态，并及时加以干预和调整。使用该系统，可以有效地提高农业大棚管理的科学性、精确性和实时性，进而提高农业的生产效益。此外，将该智能大棚接入到互联网可以实现包括农产品溯源、现实版开心农场等增值应用。</p></div> |
|--|

1.2 国内外概况和发展趋势

（1）国外研究与发展趋势

在国外，大棚种植的技术最早出现在 15 世纪到 16 世纪，此后，大棚所使用的材料不断发展，所使用的技术也不断地更新升级。大棚控制技术也随之不断发展，最早使用传感仪表对大棚中的各种参数进行采集，然后由人工根据参数反应的情况，进行手动或电动地控制执行设备，进行简单地调节和控制。这种方式效率低，往往只能改变一个环境因素，无法对影响大棚环境的参数进行综合调整，因此无法获得最佳的效果。随着技术的发展，到了上个世纪 80 年代，基于工业控制机制、可编程逻辑器件的集散控制系统开始出现，计算机被用于控制大棚中的各类环境因素的参数，这样一来，人们可以为大棚中作物的生长实施最佳的控制。此后，随着网络技术、自动控制技术和无线通信技术的发展，大棚控制系统的架构不断更新，其控制算法也不断改进，比如基于模糊控制和多变量控制^[1-2]等技术，被应用到大棚的控制之中。比如，美国在智能大棚监控技术中采用了一种集成环境调控、土壤灌溉和农作物施肥于一体的智能大棚网络监控系统，整个系统将各种生产调控进行整合，根据大棚内传感器采集的数据来控制大棚内的每个执行动作，从而使智能大棚的控制变的更加有效率，提高了投入产出比^[3]。以色列在智能大棚监控技术中使用了计算机环境监控系统，它综合了高质量外围结构的温室、温湿度和光照等环境监控系统、天窗及窗帘等可以调节光照强度的辅助调节设备等^[3]。日本在智能大棚监控技术中根据大棚内每种作物的生长特性，将不同生长阶段作物所需要的不同环境参数值输入到计算机程序中保存，在大棚的监控过程中，如果作物的一种环境参数值发生异常，则它其余的环境参数值也会根据预先计算机设定的程序来进行修改和校准，从而可以使大棚内作物的环境达到动态平衡的状态，使大棚内作物更好的生长^[3]。

目前，国外智能大棚的建设方面，已经建立了相关机构，制定了统一的规范标准，计算机网络技术、现代测控技术、专家系统和远程遥感技术等与智能大棚监控技术相结合，使得国外智能大棚控制技术进一步发展^[4-5]。今后，在控制技术的智能算法、精密控制、智能控制、多因素综合监控等，将成为发展的方向^[7]。

（2）国内研究现状与发展趋势

我国智能大棚技术的发展起步较晚，从 20 世纪 70 年代开始，才开始使用塑料温室大棚、节能日光温室大棚等的推广和发展。这种大棚的控制采用手动方式，完全依靠操作者的经验，劳动生产率低，无法进行农业生产的精确控制。20 世纪 80 年代开始，我国开始引入国外的大棚控制技术，通常都采用了计算机控制技术。从 90 年代开始，我国开始了对大棚相关设施的全面研究，智能大棚系统向着智能化、高产化的方向发展，智能大棚的使用面积也不断扩

大^[6-7]。我国许多研究机构利用先进的信息技术,对我国智能大棚监控技术进行了深入地研究,比如,中国农业科学院农业环境与可持续发展研究所可控环境农业实验室针对国内温室大棚的监控现状,开发研制了一套用于大棚内环境监测与控制的系统,整套系统采用 Internet 和 RS-485 总线技术,并在 ASP.NET 技术基础上研发了 B/S(Browser/Server)模式的远程智能大棚监控系统,整个系统投入到了实际生产中,并取得了良好的效果,提高了我国在农业大棚监测与控制技术方面的整体水平^[8]。

中国农业大学开发了一套温室环境监控系统,包括主计算机、温室监控设备和室外气象监测站三个部分。在这三个部分中,主计算机主要放置于监控中心处,对温室环境中的监控系统进行集中管理,管理内容有设置参数、执行预定的控制算法、发送控制指令及保存、查看、输出各种数据等功能。温室监控设备放在每一个温室中,它包括环境检测系统、摄像头和调节环境的执行设备,整套监控系统能够实时监测与控制温室内的环境状态^[8]。

与国外的智能大棚的发展趋势类似,国内的研究也随着计算机网络技术、无线传感网技术、自动控制技术^[11-13]等的发展,我国智能大棚的监测与控制技术将更加注重系统的精准度、实时性,并将综合考虑影响大棚状态的各种因子的调整,并不断利用新的控制理论和控制考虑,保证智能大棚的持续发展^[14-15]。

1.3 选题的先进性和实用性

本选题相对于国内已有的智能大棚的设计,除了使用了多种传感器,并使用短距离无线通信技术完成数据收集和控制命令传输之外,还实现了智能大棚网关直接接入局域网或互联网的功能,具有一定的先进性。

本选题基于无线传感网,构建智能大棚控制系统,实现大棚内各类数据的实时采集、自动控制、远程测控等功能,可以满足农业大棚的应用需要,具有一定的实用性;

1.4 选题的技术难度和工作量

本选题技术难度适中,所需技术涉及面较广,能够达到研究生论文的要求,工作量适中,能够在预定的时间内完成选题的各项研究内容。

2. 主要参考文献(列出作者、论文名称、期刊名称、出版年月)。

[1] 王杰,管涛,李玉玲. 基于 SSH 和模糊控制的精准农业大棚智能监控平台[J]. 湖北农业科学, 2014,53(15): 3650-3658.

[2] Ma Juncheng, Li Lin, Liu Xue, Fu Zetian, Zhang Lingxian. A fuzzy comprehensive evaluation of the applicability of intelligent greenhouse control systems[J]. Sensor Letters, 2013,11(6-7): 1396-1402.

批注 [yy1]: 不能凭主观认为,要客观描述技术难度和工作量表现在哪里

批注 [yy2]: 这些文献的引用都应在正文中逐一标注。

- [3] 谭剑. 基于 WSN 的智能农业大棚设计与实现[D]. 扬州: 扬州大学, 2013.
- [4] 李培. 基于 WSN 的智能温室大棚自动定点喷灌系统[J]. 农机化研究, 2014(7): 76-79.
- [5] 陶平. 基于 ZigBee 的温室大棚智能监控系统研究[D]. 成都: 西华大学, 2012.
- [6] 张守艳. 基于 ZigBee 的智能大棚控制系统研究[D]. 曲阜: 曲阜师范大学, 2014.
- [7] 鞠传香, 吴志勇. 基于 ZigBee 技术的温室大棚智能监控系统[J]. 江苏农业科学, 2013, 41(12): 405-407.
- [8] 豁保强. 智能大棚监测与控制关键技术的研究[D]. 天津: 天津科技大学, 2014.
- [9] Qiu Weimin, Dong Linxi, Wang Fei, Yan Haixia. Design of intelligent greenhouse environment monitoring system based on ZigBee and embedded technology[C]. 2014 IEEE International Conference on Consumer Electronics - China, ICCE-C 2014, pp. 1-3.
- [10] Gang Dong. Intelligent greenhouse control system design based on rf technology research[J]. Applied Mechanics and Materials, 2014, 687-691: 64-67.
- [11] 孙彦景, 丁晓慧, 于满, 田红. 基于物联网的农业信息化系统研究与设计[J]. 计算机研究与发展, 2011, 48(suppl.): 326-331.
- [12] 颜波, 石平. 基于物联网的水产养殖智能化监控系统[J]. 农业机械学报, 2014, 45(1): 259-265.
- [13] 朱勇, 卿培. 基于新簇路由算法的温室监控网络节能研究[J]. 仪器仪表学报, 2012, 33(6): 1420-1426.
- [14] 吴舟. 基于移动互联网的农业大棚智能监控系统的设计与实现[D]. 北京: 北京邮电大学, 2013.
- [15] 刘力, 鲍安红, 曹树星, 胡秀芝. 温室大棚内环境自动化控制方案设计[J]. 农机化研究, 2013(1): 90-93.

三、课题内容及具体方案

1. 课题内容

本课题主要基于无线传感网技术，在研究相关理论和技术的基础上，设计和实现智能大棚控制系统，主要的内容包括以下部分：

- （1）研究农业用传感器技术、无线传感网技术、智能控制算法、嵌入式技术等理论知识，为系统的设计与实现奠定基础。
- （2）对系统进行需求分析，绘制系统总体拓扑结构图，明确课题的实现目标。
- （3）完成系统的硬件平台设计和实现，包括数据采集节点、执行设备节点、网关节点等的硬件设计。
- （4）完成系统的软件设计与实现，包括节点及网关的软件设计，特别是网关的智能控制算法、web 服务器的设计等。
- （5）对所设计和实现的系统进行全面测试和改进，提高系统的稳定性和可靠性。

2. 系统需求分析（应用软件工程专业描述工具描述）

2.1 概述

智能大棚可满足精细农业生产的需要，实现对农业生产过程、作物生长环境的监测和控制，需要借助各类传感器以及无线网络等共同作用，才能完成所需的功能。软件的操作界面应该简单，使用方便，应具备较强的数据独立性，确保在软、硬件环境发生变化时，数据的完整、安全及有效利用。

2.2 功能需求

- （1）为用户提供直观、简介、方便的操作界面，用户可以通过 GUI 界面，实时查看大棚的环境状况，了解作物生长的土壤、空气、温度、湿度等数据，可以对大棚内操作部件进行控制。用户还应能通过 GUI 界面，设置与大棚所种植作物相适应的参数。
- （2）系统应当包含数据采集节点，完成大棚内温度、湿度、气体浓度、光照强度、土壤湿度、土壤营养元素等数据的采集。
- （3）系统应当包含执行部件，完成浇灌、通风、施肥、增加光照等操作，为作物生长创造优良的环境。
- （4）数据采集节点应当分散部署到大棚的不同位置，以便能够让它们采集的数据可以客观地反应大棚内的实际状况。
- （5）系统应当使用短距离无线通信技术来实现数据的传输，并考虑终端节点的供电情况，还应为数据的有效传输定义通信协议。
- （6）系统的网关应当提供接入局域网或互联网的功能，并设计 Web 服务器，供用户实现远

批注 [yy3]: 这里的需求还是停留在用户需求描述，需要转换成系统设计的需求，并用 UML 等工具描述系统结构，各个模块的控制过程和数据处理过程，下一步的概要设计才能有合理性。建议仔细研究一下软件工程。

程测控的需要。网关应当能够根据用户设置的参数，按照智能算法实现自动化控制的功能。

2.3 性能需求

(1) 精度

①系统响应用户从 GUI 界面输入的指令，时间不能多于 0.2 秒，对于系统运行负荷较重时，该响应时间也不能超过 0.5 秒。

②数据在节点与网关之间传输，要求传输时间不能超过 1 秒。

③在网关接收到用户输入指令之后，向执行部件发出动作命令之后，执行部件应当在 2 秒之内给予响应。

④ 温度传感器采集的温度误差应不超过 1%，在本系统当中温度的显示范围为零下 10 至 50 摄氏度。

⑤无线传输的数据包正确率应保证在 90%以上，并提供检错机制，保证通信的有效性。

(2) 灵活性

①当在用户界面输入恶意破坏性命令时，系统不做处理；

②当运行环境发生变化后，本系统仍然可以保持较好的运行效果；

③本项目对软件要求比较低，兼容性较好；

④因本产品的使用性质决定了对系统的反映（处理数据与命令控制）性能要求比较高，处理速度比较快；

⑤当用户在操作本产品的时候由于突发情况是网关断电后，应能保证数据不会丢失，当再一次上电以后仍然可以查阅之前的数据。

2.4 接口需求

本系统应提供用户输入接口，即 GUI 界面，用户可以在网关操作界面，按下相关的命令，系统做出相应的反应。网关还应提供网络接口，对内使用无线通信接口，对外提供以太网接口。

2.5 其他需求

(1) 数据管理能力要求

网关应当提供数据存储功能，包括用户设置的参数、操作日志、系统异常状况记录等，并能够提供查询功能。其中，用户还能够通过 GUI 界面设置所需的参数。

(2) 故障处理要求

当系统处于不稳定状态和死机状态，可以通过手动重启方式恢复系统初始化，如果恢复不能成功，则需要本系统开发技术人员进行升级维护。

(3) 安全性要求

要求系统具有安全、可靠、可维护，提供保证安全的错误，如身份验证、加密保护等措施。

根据上述系统需求分析，可以得到图 2-41 到图 2-33 的网关用例图、传感器节点用例图和执行部件用例图。

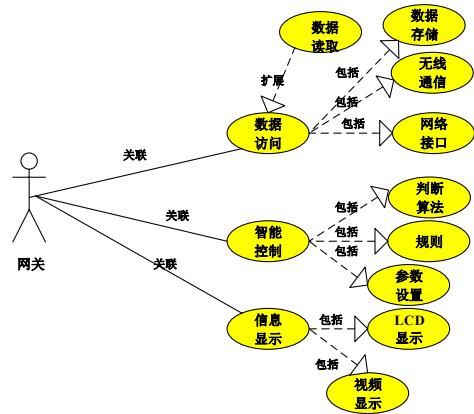


图 2-41 网关用例图

批注 [yy4]: 图顺序编号即可 (图 1、图 2...)

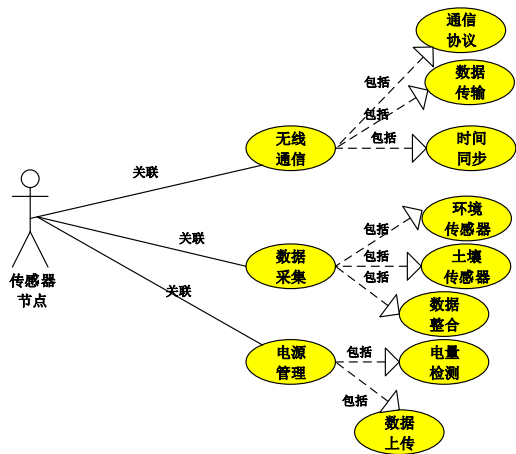


图 2-42 传感器节点用例图

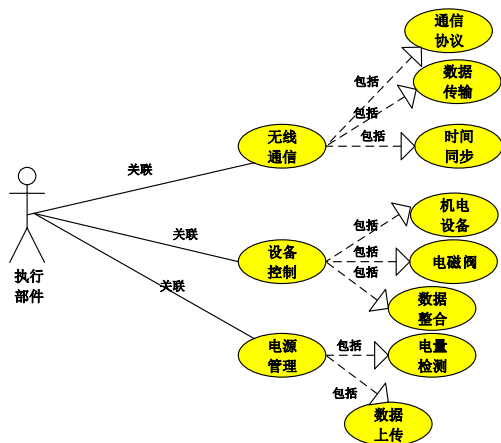


图 2.3-3 执行部件用例图

3. 系统概要设计（应用软件工程专业描述工具描述）

3.1 系统拓扑结构

系统的拓扑结构如图 3.1-4 所示。在智能大棚中，多处设置传感器节点及执行部件，其中传感器节点主要负责采集各类环境及土壤数据，执行部件则主要完成各种操作，比如浇灌、施肥等。每个节点都使用了无线通信的方式与网关进行通信。大棚内还配备了摄像头，并与网关相连，可通过网关实时查看大棚内的状况。网关还提供了对外的接口，可以直接接入网络，供远端的用户通过 PC 终端、平板电脑、智能手机等终端访问。

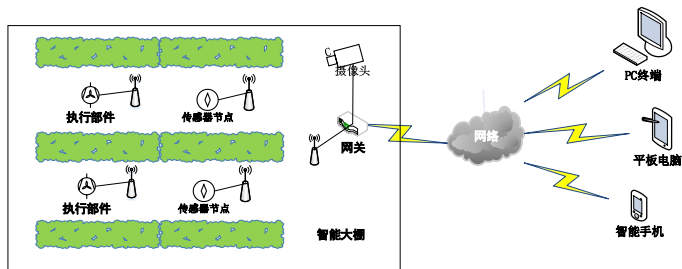


图 3.1-4 系统拓扑结构图

3.2 系统硬件总体设计

(1) 网关硬件设计

批注 [yy5]: 本部分要到概要设计的程度。描述方法要专业，应尽量用UML的方法描述。

批注 [yy6]: 标题不要在一页的最后一行

网关的硬件框图如图 3-25 所示。

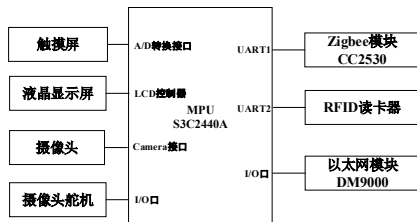


图 3-25 网关硬件框图

从图中可以看出，网关选用的处理器为 S3C2440A，该处理器基于 ARM920T 核心。处理器外围通过串口（UART）连接了 Zigbee 模块和 RFID 读卡器，可用于节点间的通信，以及门禁系统的身份验证。处理器还连接了触摸屏及液晶屏，可以用于显示信息及提供用户输入信息。处理器通过专用的 camera 接口连接了摄像头，同时还通过 I/O 口连接了控制摄像头转动的舵机。为了实现接入网络的功能，网关还连接有以太网模块。

（2）节点硬件设计

终端节点分类两类，即传感器节点及执行部件，他们的结构分别如图 3-36 和图 3-47 所示。



图 3-36 传感器节点硬件框图

从图 3-36 可以看到，传感器节点使用了 CC2530 作为主控芯片，该芯片集成有 51 单片机及 Zigbee 射频通信模块，可以实现对节点数据采集、控制和通信的功能。通过 A/D 转换接口，CC2530 与土壤水分传感器、土壤温度传感器、气体浓度传感器、光照强度传感器相连接，温湿度传感器由于输出的信号为数字信号，因此直接通过 I/O 口与 CC2530 相连。

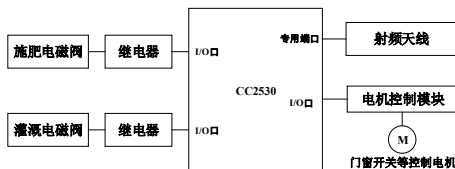


图 3-47 执行部件的硬件框图

执行部件的处理器也选用了 CC2530，通过 I/O 控制所连接的继电器，可以控制施肥电磁阀及

灌溉电磁阀的开关，从而实现液态养料及水的供给；通过 I/O 口还可以通过电机控制模块，完成对大棚内各种电机的控制，主要包括门禁电机、通风扇、透光窗等设备。

3.3 系统软件总体设计

(1) 嵌入式实时操作系统选型及移植

嵌入式系统由于需要较高的可靠性和实时性，采用嵌入式实时操作系统可以有效地利用其提供的平台，提高代码的可靠性，缩短开发的周期。在本课题中，选用 $\mu\text{C}/\text{OS-II}$ 操作系统。 $\mu\text{C}/\text{OS-II}$ 由美国 Micrium 公司发布，最早是由 Jean J.Labrosse 开发。 $\mu\text{C}/\text{OS-II}$ 是基于优先级的抢占式实时操作系统，系统可以采用多任务的机制工作。 $\mu\text{C}/\text{OS-II}$ 是一个完全开源的操作系统，而且配有详细的使用手册，其实时性还获得了美国联邦航空管理局的 RTCA DO-178B 标准的认证，可用于安全要求非常苛刻的系统之中。

图 3-58 给出了 $\mu\text{C}/\text{OS-II}$ 的启动流程图，从图中可以看出， $\mu\text{C}/\text{OS-II}$ 的应用程序开始于 main() 函数，首先完成处理器的初始化，然后初始化硬件平台的各种模块；接着调用 OSinit() 函数初始化操作系统，之后进行任务创建， $\mu\text{C}/\text{OS-II}$ 要求在启动多任务机制之前，必须至少创建了一个用户任务；最后，调用 OSStart() 函数，启动多任务机制。

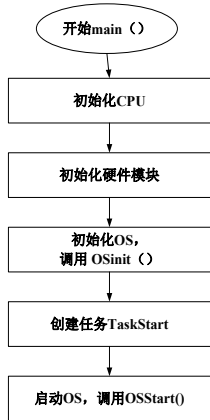


图 3-58 系统启动流程

$\mu\text{C}/\text{OS-II}$ 为应用软件提供了一个多任务并发运行的平台，应用软件的开发主要是将功能分离成多个可以并发运行的任务，多个任务之间通过互斥机制使用资源，通过同步机制实现合作。 $\mu\text{C}/\text{OS-II}$ 在运行时，会对多个任务基于优先级地调度，优先级高的任务将优先获得运行，除非高优先级的任务主动放弃 CPU，或由于等待某事件的发生，才会失去 CPU 的使用权。因此，如何设

定任务的优先级，对于充分发挥多任务机制的优势，有着重要的意义。

(2) 网关软件多任务结构

智能大棚系统的网关主要起到数据的收集、处理和存储的工作，可以接收传感器节点的数据，控制执行部件工作，响应来自网络请求，据此可以得到图 3.69 所示的数据流程图。

批注 [yy7]: 图要清晰一些

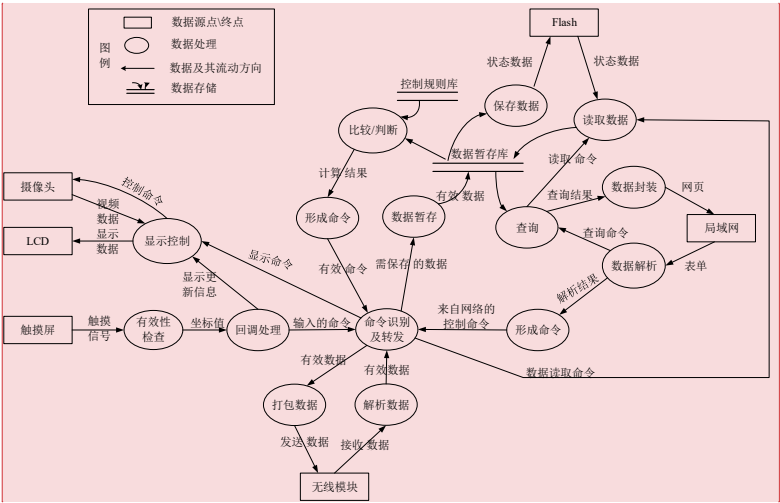


图 3.69 网关数据流程图

从图 3.6 可以看出，触摸屏输入的触摸信号，经过有效性检查之后经回调处理之后，形成输入命令，同时，回调处理会向 LCD 发送需要更新的信息，摄像头的显示也根据控制命令，显示在 LCD 之上；当来自无线模块的数据被接收及解析之后，将被识别出其中包含的数据和命令，并将数据存储到暂存库中，这些数据会被系统在参照规则的基础上，进行比较/判断，然后将结果形成命令，完成命令后被转发到无线模块发送出去；来自局域网的查询要求，会被解析，形成命令，并在完成查询之后，生成动态网页，发给客户端；此外，保存在暂存库中的数据会被定时保存到 Flash 之中。

在本课题所设计系统中，网关的工作基于嵌入式实时操作系统，采用多任务的形式设计应用软件，并提供图形界面与用户进行交互。既可以保证系统的实时性要求，又能方便用户的操作，网关的多任务结构如图 3.710 所示。

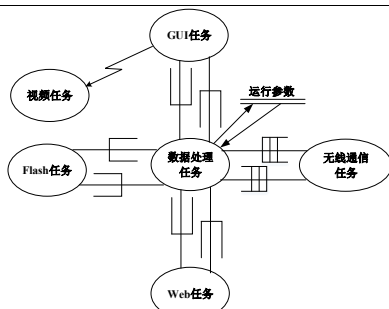


图 3-710 网关的多任务结构图

网关共六个任务，即数据处理任务、无线通信任务、Web 任务、Flash 任务、GUI 任务和视频任务。其中数据处理任务处于中心位置，可以接收来自 GUI 任务用户的输入信息，也可以向 GUI 任务发送需要更新的显示内容；当数据处理任务接收到来自无线通信任务的数据之后，会依据通信协议解析数据，根据这些数据，结合用户设置的运行参数判断系统状态，如果需要对当前的状态进行调节，则通过无线通信任务，向执行部件发送命令；数据处理任务还可以将需要长期保持的数据周期性地发送给 Flash 任务，由它保持到 Flash 中，需要查询的时候，Flash 任务再将数据读出发送给数据处理任务；Web 任务会处理来自网络请求，如果需要读取当前的传感器数据，它会将请求发送给数据处理任务，由数据处理任务完成数据的读取工作，然后交由 Web 任务发送给远程用户。此外，当使用者通过 GUI 界面启动了视频，则 GUI 任务会将挂起的视频任务激活，进行视频显示。

（3）通信协议设计

在本课题中，采用了基于 Zigbee 的无线通信技术，要实现终端节点与网关之间的通信，需要在 Zigbee 的协议栈 Z-stack 之外，还需要自定义应用层通信协议，主要包括以下内容：

①基本原理

网关的 Zigbee 模块与 ARM 处理器之间，采用 RS232 全双工、无流控的全双工通信方式；每字符采用 1 比特起始、8 比特信息、无校验和 1 比特停止位；通信速率为 115200bps。

无线通信采用超时定时发送的方式。当终端节点（传感器节点和控制节点）在上电有连接状态改变时或路由节点超时发送路由信息。当终端节点超时发送传感器采集数据。

②传感器数据帧格式：

Byte0: 0x2 （帧头）

Byte1: len （包长度, Byte2 到 FCS 前的字节数。即 $n-2+1$ ，或有效数据长度+6）

Byte2、3: 0x46B9 (cmd, 控制命令,低字节在前)

Byte4: 0xF1 (cmdEndPoint, 命令端节点号)

Byte5、6: SrcShortAddr (两个字节源短地址, 低字节在前, 网关上电时为 0x0000)

Byte7: 任务号(一般为 1)

Byte8、9: ParentShortAddr (两个字节父短地址, 低字节在前, 网关为 0x0000)

Byte10: 节点及传感器板类型号
 D7D6 为节点类型(00: 网关节点; 01: 路由节点; 10: 端节点)
 D5D4D3D2D1D0 为传感器板类型代码(从 0x0~0x9)

Byte11: 终端节点上 A/D 采集值。0 为 0V; 0x7F 为 3.3V。

Byte12: 土壤水分值

Byte13: 土壤温度值

Byte14: 温湿度传感器温度, 单位为℃

Byte15: 温湿度传感器湿度, 单位为%

Byte16: 光照度。0~0xFF

Byte17: 湿度值

Byte18: 气体浓度值

Byte19: FCS(Byte1~18 的字节异或值)

③控制节点数据帧格式

Byte0: 0x2 (帧头)

Byte1: len(包长度, Byte2 到 FCS 前的字节数。即 n-2+1, 或有效数据长度+6)

Byte2、3: 0x46B9 (cmd, 控制命令,低字节在前)

Byte4: 0xF1(cmdEndPoint, 命令端节点号)

Byte5、6: DstShortAddr(两个字节短地址, 低字节在前, **网关上电时为 0x0000**)

Byte7: 任务号(一般为 1)

Byte8: 控制节点编号

Byte9~n: 传送的控制数据

Byte10: FCS(Byte1~n 的字节异或值)

(4) 传感器节点软件设计

传感器节点负责采集各种传感器的数据, 然后按照本课题所设计的通信协议打包成数据帧,

在通过 Zigbee 技术发送给网关，传感器节点可以处于多种工作状态，如 3.8.11 给出了其状态图。

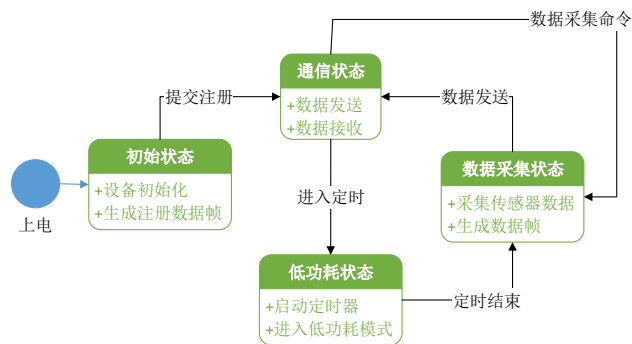


图 3.8.11 传感器节点的状态图

传感器节点在上电之后，将进入初始化状态，完成设备初始化及生成向网关注册的数据帧，当将注册数据帧提交注册后，切换到通信状态，完成数据的发送，然后进入定时，启动定时器，系统进入低功耗状态，定时结束之后，系统切换到数据采集状态，对采集到的数据生成数据帧之后，提出数据发送操作之后，进入通信状态，系统发送数据；当系统接收到数据时，系统会识别相应的命令，切换到数据采集状态，进行数据采集。

传感器节点的初始化流程如图 3.9.12 所示。传感器节点的软件开发基于 TI 公司的 Zigbee 协议栈开发，该协议栈设置了一个操作系统抽象层，可实现带优先级的多任务轮转算法。在本系统中，只需要在 Zigbee 协议栈启动时，运行初始化程序，然后使用定时器来完成其他工作。

节点首先初始化 I/O 口，使得每个外部设备都能够与 CC2530 正常连接；然后，初识化 A/D 转换，以方面后续的数据采集；接着初始化缓冲区，读取配置信息，检测所连接的每个传感器的状态，再接着依据检测的结果向网关注册这些信息，以便网关可以知道当前哪些传感器是正常的，哪些是不能正常工作的；在收到来自网关的注册成功的反馈信息之后，节点记录已注册传感器到设备信息表中；最后，则启动定时器，等待定时时间的到来。

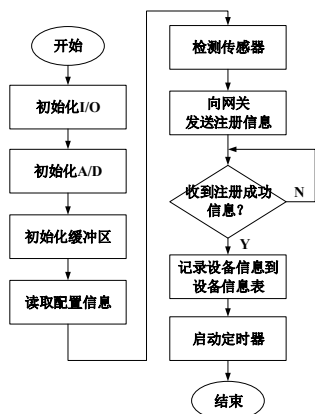


图 3-912 传感器节点初始化流程

定时器的作用主要是用于周期性地采集传感器的数据，然后发送给网关，定时的间隔时间长短，与传感器的类型有关，还与网关处理算法相关，具体要根据需要来进行调整，其处理流程如图 3-4013 所示。

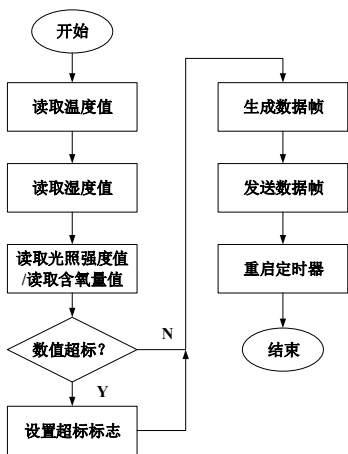


图 3-4013 传感器节点定时处理流程图

(5) 执行部件软件设计

执行部件工作中，与传感器节点类似，也可以处于多种状态，图 3.4414 给出了它的状态图。

执行部件上电之后，首先处于初始化状态，完成设备的初始化和生成注册数据帧，提交注册后，转换到通信状态，在发送完注册数据帧之后，将进入睡眠，切换到低功耗状态，关闭外部设备，在被唤醒后，主要是接收到了来自网关的数据，见进入控制状态，可以执行网关发来的控制命令，操作完成之后，形成反馈数据帧，切换到通信状态，发往网关。

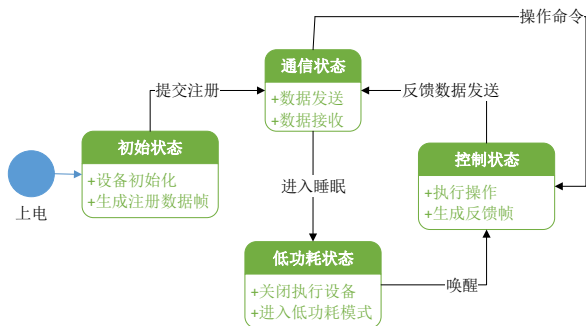


图 3.41-14 执行部件的状态图

执行部件主要根据网关的控制命令，启动或关闭各类执行设备，如电机、电磁阀等。与传感器节点类似，执行部件也是基于 TI 公司的 Zigbee 协议栈来开发软件。由于执行部件主要被动接受来自网关的命令，因此，主要在 Zigbee 协议栈已有的任务框架下，重写接收到数据后的处理函数，其流程如图 3.42-15 所示。

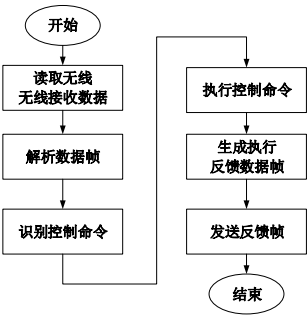


图 3.42-15 执行部件的处理流程

执行部件在接受到了无线数据之后，首先读取这些数据，然后将其解析，识别其中的控制命令，然后执行命令，并将执行的结果生成反馈数据帧，发往网关。

4. 拟采用的开发方法、环境，测试方案等

本课题所开发系统涉及硬件和软件部分，分别采用以下的开发方法、环境和测试方案：

（1）硬件部分

对于节点部分，需要使用 Altium Designer 绘制原理图以及 PCB 图，使用万用表、示波器、逻辑分析仪等对 PCB 板进行硬件测试；然后，编写底层驱动，测试 PCB 板的工作情况；最后，根据测试结果，修改和优化电路设计。对于网关部分，拟选取典型的嵌入式系统开发平台，以达到在满足功能要求的前提下，节省开发时间，提高系统稳定性。

（2）软件部分

软件部分的开发属于嵌入式软件及基于 Zigbee 协议栈的开发，因此，对于节点主要是基于 Zigbee 协议栈，围绕 CC2530 来进行软件开发，使用 IAR 集成开发环境，所开发出的软件需要按照软件测试的方法，完成功能测试、性能测试等；网关端的软件也需要使用 Zigbee 模块，但仅是将其作为网关的一个部分，因此，网关的软件开发主要集中在基于嵌入式操作系统的开发之上，包括智能大棚控制软件、Web 服务器等内容。对网关的测试也要完成功能测试和性能测试，特别是控制算法的有效性、效率，Web 服务器的效率、吞吐量、安全性等的测试。此外，还需要设计节点与网关之前的通信协议，以保证信息的透明高效传输。

5. 技术难度及特色分析

本课题涉及传感器技术、Zigbee 技术、嵌入式操作系统、通信协议、智能算法、Web 服务器技术等，涉及的技术面较广，技术难度主要存在于以下几个方面：

（1）大棚控制的智能算法的设计

大棚应当允许用户针对大棚所栽培作物，设定相应的控制程序，根据作物习性，设定控制参数，达到智能控制的效果。因此，需要研究如何实现这些功能，并能够更加精确地完成大棚根据不同作物的精细测控。

（2）自定义通信协议的设计

网关和传感器节点、执行部件之间通过 zigbee 技术进行通信，其软件开发应当给予 zigbee 协议栈，但还需要定义应用层协议，因此，课题的一个难点和重点就是定义通信协议，并测试该协议，使之能够适应通信的需要。

（3）系统通信的稳定性和可靠性

系统采用的是集散式的结构，采用无线通信技术，节点分布在大棚的不同位置，如何保证系统稳定性和数据传输的可靠性是课题研究的一个重点和难点。

这些技术都有丰富的参考资料，通过与实际需求相结合，进行深入研究，可以加以解决。

本课题所设计系统具有如下的特色：

批注 [yy8]: 技术难点要展开论述，不能笼统。

（1）系统将大棚中所包含的各种环境数据，进行了全面采集，具有很强的综合性，数据可以实时传输、显示和应用，具有实时性；

（2）系统采用了无线通信技术，摒除了有限网络带来的布线困难、维护繁琐等的缺点，提供了管理效率和管理效果；

（3）系统具备接入局域网或互联网的功能，为大棚的远程测控提供了接口；

（4）系统的网关部分采用了多任务机制设计，具有较强的实时性，可以满足实际生产的需要，具有较高的响应和处理速度。

6、本人主要工作描述

在本课题中，本文需要完成对相关理论的研究，完成系统总体架构的设计，对处理器、专用模块、元器件等进行选型，完成硬件电路的测试，完成软件的设计、编写和测试的工作。

四、工作进度的大致安排

应包括文献调研，工程设计，项目开发和调试，实验数据的分析处理，撰写论文等。

| 时间 | 工作 | 阶段成果 |
|-----------------|--------------------------------------|---------------|
| 2015.10-2015.11 | 完成相关技术和文献资料的研究 | 文献综述，开题报告 |
| 2015.11-2015.12 | 完成系统的需求分析，并规划项目总体框架的设计 | 需求分析报告 |
| 2016.01-2016.03 | 选取硬件相关部件，完成系统软硬件的概要设计 | 概要设计报告 |
| 2016.04-2016.06 | 完成硬件电路的设计和 PCB 制作，并进行测试；完成软件部分的详细设计。 | 详细设计报告，硬件电路板 |
| 2016.07-2016.09 | 完成软件代码的编写，并与硬件平台进行集成测试，并根据测试结果优化设计。 | 项目代码，完整的软硬件系统 |
| 2016.10 | 整理相关文档，撰写毕业论文，进行毕业论文答辩 | 毕业论文初稿 |
| 2016.11 | 修改和完善毕业论文 | 毕业论文定稿 |

五、预期成果

应包括软硬件产品、文档、模型、专利、论文等

- (1) 完成基于无线传感网的智能大棚系统的设计与实现，包括其硬件部分及软件。
- (2) 总结研究和设计成果，撰写工程硕士学位论文。